



云计算 (第三版)

CLOUD COMPUTING Third Edition

第2章

Google云计算原理与应用 (二)

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.3 分布式锁服务Chubby

• 初步了解Chubby

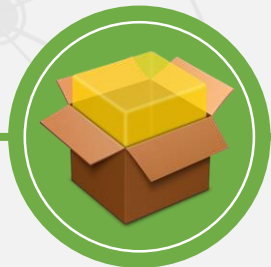
Chubby是Google设计的提供**粗粒度**锁服务的一个文件系统，它基于**松耦合**分布式系统，解决了分布的**一致性**问题。

Chubby是一种**建议性**的锁，而非强制性的锁，使得系统更灵活。

GFS使用Chubby选取一个GFS主服务器，Bigtable使用Chubby指定一个主服务器并发现、控制与其相关的子表服务器。



通过使用Chubby的**锁服务**，用户可以确保数据操作过程中的**一致性**



Chubby还可以作为一个稳定的**存储系统**存储包括元数据在内的小数据



Google内部还使用Chubby进行**名字服务** (Name Server)

2.3 分布式锁服务Chubby

- ▶ 2.3.1 Paxos算法
- 2.3.2 Chubby系统设计
- 2.3.3 Chubby中的Paxos
- 2.3.4 Chubby文件系统
- 2.3.5 通信协议
- 2.3.6 正确性与性能

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

Chubby内部的一致性问题的实现用到了Paxos算法。

Paxos算法是一种基于消息传递的一致性算法，用于解决分布式系统中的一致性问题。

简单地说，分布式系统的一致性问题，就是如何保证系统中初始状态相同的各个节点在执行相同的操作序列时，看到的指令序列是完全一致的，并且最终得到完全一致的结果。

怎样才能保证在一个操作序列中每个步骤仅有一个值呢？

一个最简单的方案就是设置一个**专门节点**，在每次需要操作前，系统的各部分向它发出请求，告诉该节点接下来系统要做什么。

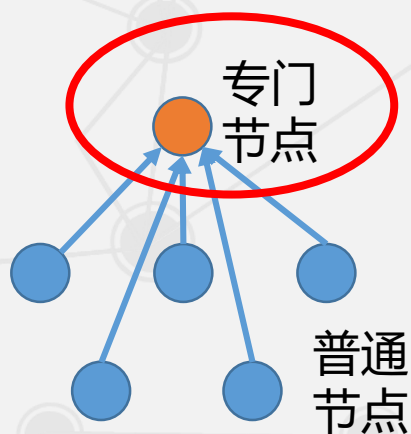
该节点接受第一个到达的请求内容作为接下来的操作，这样就能够保证系统只有一个唯一的操作序列。

2.3 分布式锁服务Chubby

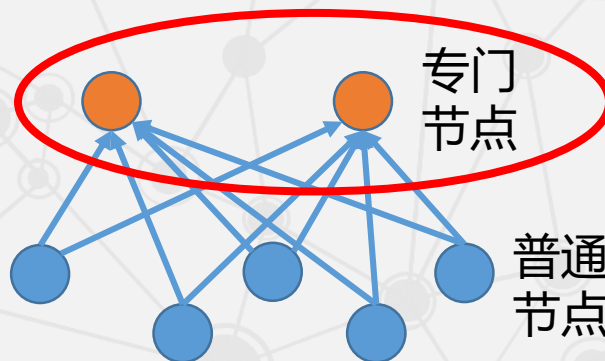
• 2.3.1 Paxos算法

但设置专门节点的方案有一个明显的缺陷，就是一旦这个专门节点**失效**，整个系统就很可能出现不一致。

为了避免这种情况，在系统中必然要设置**多个**专门节点，由这些节点来**共同决定**操作序列。这就需要考虑**一致性问题**。



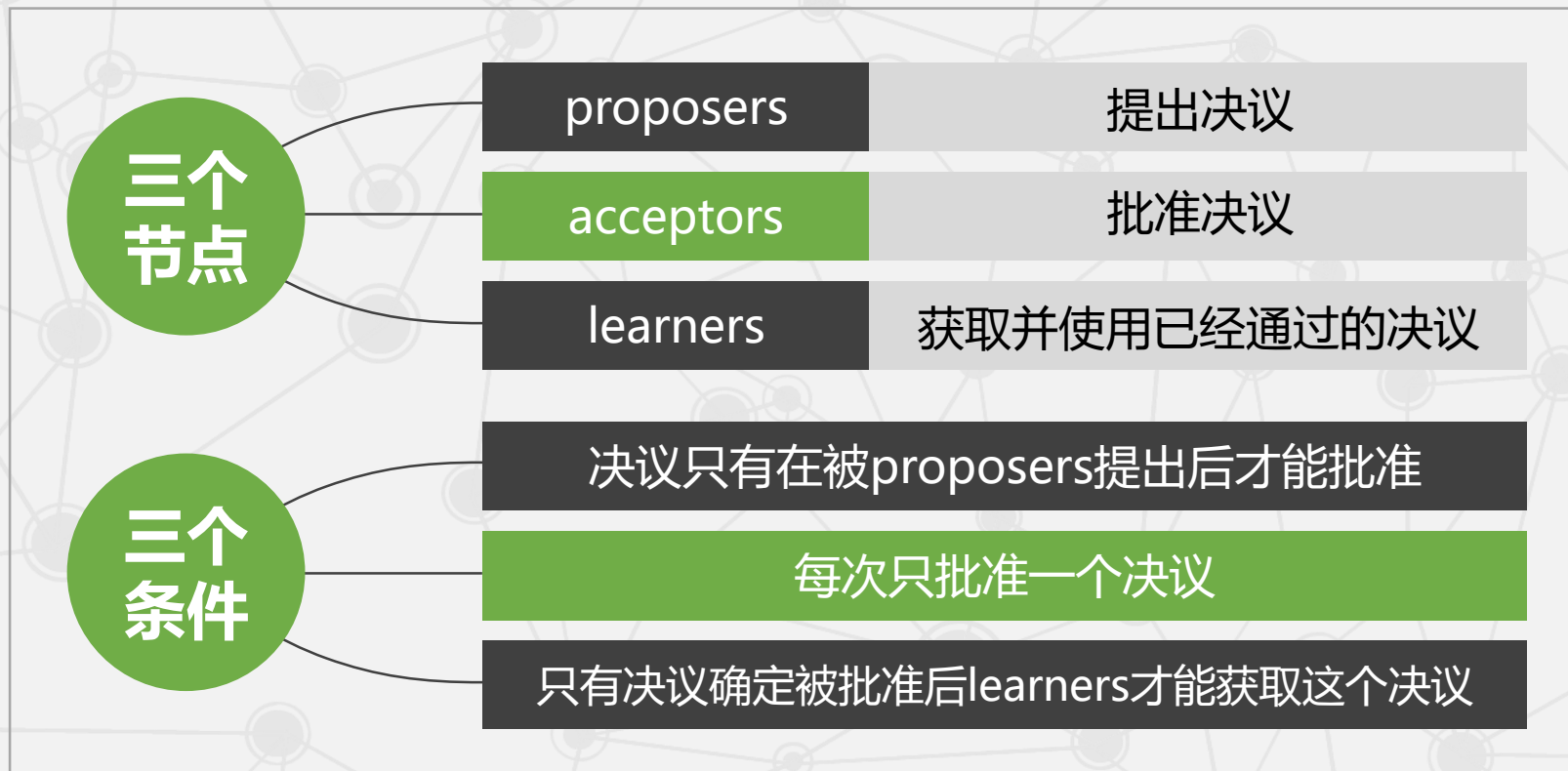
单点失效
问题



一致性
问题

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法



2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

三个条件

决议只有在被proposers提出后才能批准

每次只批准一个决议

只有决议确定被批准后learners才能获取这个决议

为了满足上述三个条件（主要**第二个**条件），必须对系统有一些约束条件。通过约束条件的不断加强，最后得到了一个可以实际运用到算法中的完整约束条件。

那么，如何得到这个完整的约束条件呢？

2.3 分布式锁服务Chubby

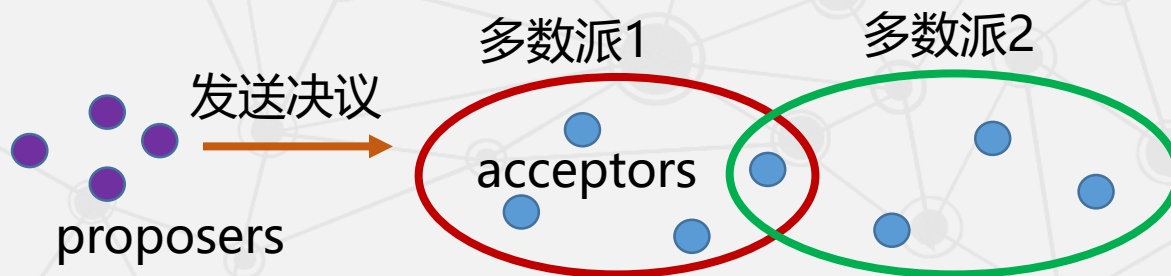
• 系统的约束条件

第二个条件：每次只批准一个决议

在决议过程中，proposers将决议发送给acceptors，acceptors对决议进行批准，**批准后的**决议才能成为正式的决议。（怎样批准？）

决议的批准采用**少数服从多数**原则，即大多数acceptors接受的决议将成为最终的正式决议。（什么是“多数派”？是否可能存在两组或多组“多数派？” 为什么会存在多组“多数派”？）

从集合论的观点来看，两组“多数派”至少有一个**公共的**acceptor。（这个公共的acceptor投了两票。）



2.3 分布式锁服务Chubby

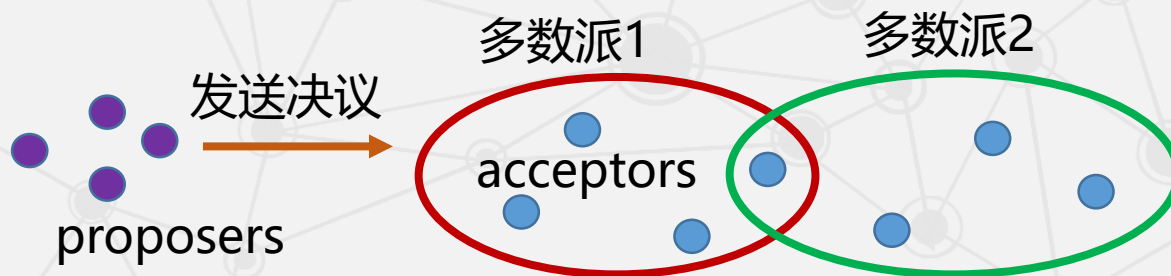
- 系统的约束条件

第二个条件：每次只批准一个决议

怎样避免出现多个“多数派”？

需要限制每个acceptor**只能接受一个决议**，则第二个条件就能够得到保证，因此，不难得到第一个约束条件：

p1：每个acceptor只接受它得到的第一个决议。（“第一个”具有唯一性）



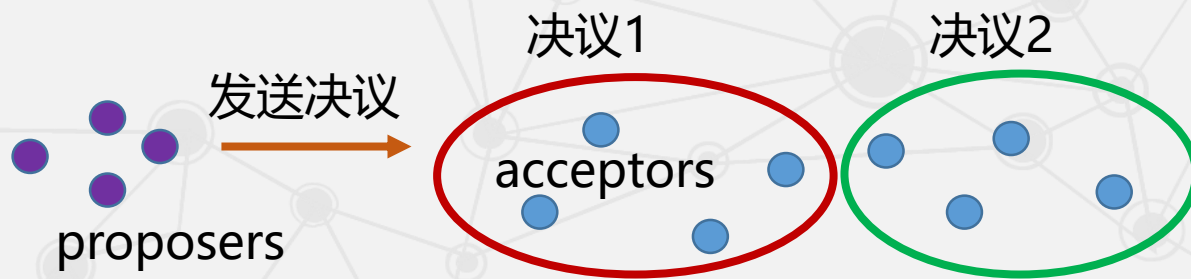
2.3 分布式锁服务Chubby

● 系统的约束条件

p1: 每个acceptor只接受它得到的第一个决议。

问题1: p1表明一个acceptor可以收到多个决议, 怎样区分“第一个”决议?
对每个决议进行编号, 后到的决议编号大于先到的决议编号。

问题2: 假设系统中一半的acceptors接受了决议1, 剩下的一半接受了决议2. 此时, 仅靠约束p1是根本无法得到一个“多数派”, 从而无法得到一个正式的决议。怎么办?



2.3 分布式锁服务Chubby

• 系统的约束条件

第二个条件：每次只批准一个决议

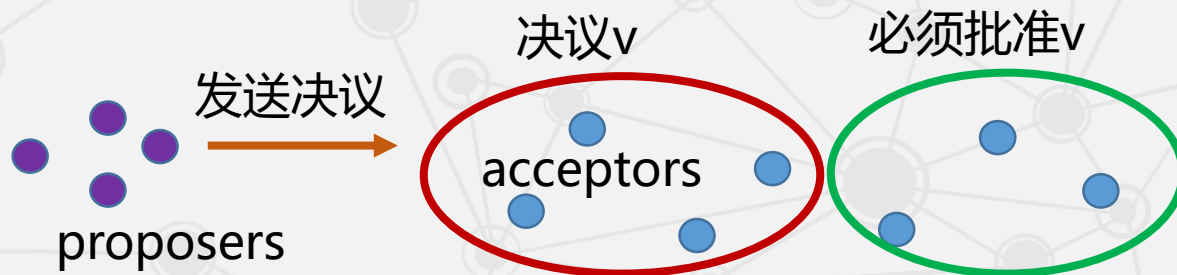
进一步加强约束得到：

p2：一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

p1和p2能够保证第二个条件。

对p2稍作加强：

p2a：一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。



2.3 分布式锁服务Chubby

• 系统的约束条件

表面上看起来已经不存在什么问题了，但实际上p2a和p1是有矛盾的。

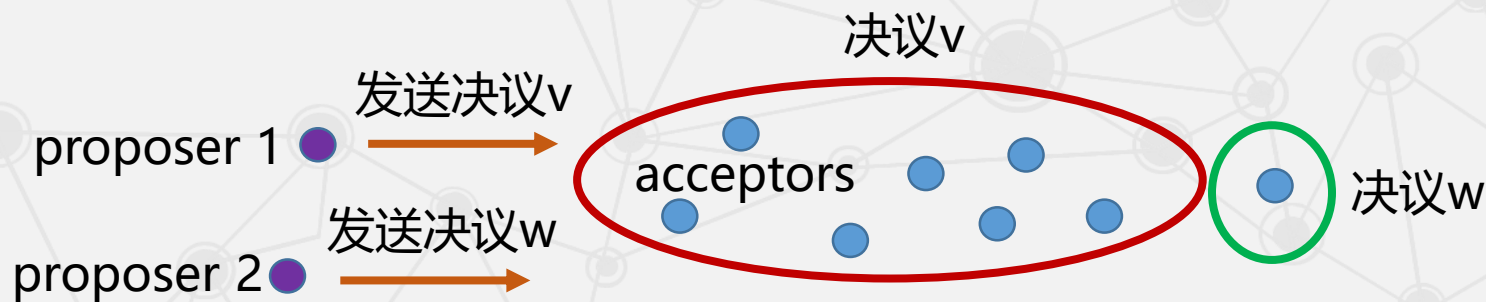
p1: 每个acceptor只接受它得到的第一个决议。

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

考虑下面这种情况：

假设在系统得到决议v的过程中，一个proposer和一个acceptor因为出现问题，并没有参与到决议的表决中。在得到决议v之后，它们才恢复过来。

此时，这个proposer提出一个决议w ($w \neq v$) 给这个acceptor。



2.3 分布式锁服务Chubby

• 系统的约束条件

第二个条件：每次只批准一个决议

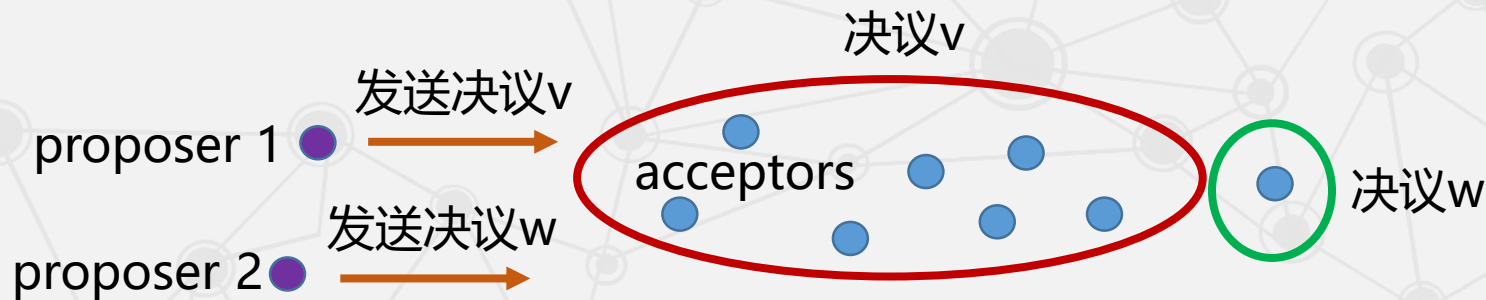
p1: 每个acceptor只接受它得到的第一个决议。

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

如果按照p1，这个acceptor应该接受决议w，但是按照p2a，则不应该接受这个决议。所以，还需进一步加强约束条件：

p2b: 一旦某个决议v得到通过，之后任何proposer再提出的决议必须是v。

满足p1和p2b就能够保证第二个条件，而且彼此之间不存在矛盾。



2.3 分布式锁服务Chubby

• 系统的约束条件

第二个条件：每次只批准一个决议

p1: 每个acceptor只接受它得到的第一个决议。

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

p2b: 一旦某个决议v得到通过，之后任何proposer再提出的决议必须是v。

但是p2b很难通过技术手段实现，因此提出了一个蕴含p2b的约束条件p2c:

p2c: 如果一个编号为n的提案具有值v，那么存在一个“多数派”，要么它们中没有谁批准过编号小于n的任何提案，要么它们进行的最近一次批准的提案值为v。

为了保证决议的唯一性，acceptors也要满足一个约束条件：当且仅当acceptors没有收到编号大于n的请求时，acceptors才批准编号为n的提案。



2.3 分布式锁服务Chubby

- 在这些约束条件的基础上，一个决议可以分为两个阶段：

1

准备阶段

proposers选择一个提案并将它的编号设为n

将它发送给acceptors中的一个“多数派”

acceptors 收到后，如果提案的编号大于它已经回复的所有消息，则acceptors将自己上次的批准回复给proposers，并不再批准小于n的提案。

2

批准阶段

当proposers接收到acceptors 中的这个“多数派”的回复后，就向回复请求的acceptors发送accept请求，在符合acceptors一方的约束条件下，acceptors收到accept请求后即批准这个请求。

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

三个
节点

proposers

acceptors

learners

为了减少决议发布过程中的消息量，acceptors将这个通过的决议发送给learners的一个子集，然后由这个子集中的learners去通知所有其他的learners。

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

三个
条件

决议只有在被proposers提出后才能批准

每次只批准一个决议

只有决议确定被批准后learners才能获取这个决议

p1: 每个acceptor只接受它得到的第一个决议。

p2: 一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

p2b: 一旦某个决议v得到通过，之后任何proposer再提出的决议必须是v。

p2c: 如果一个编号为n的提案具有值v，那么存在一个“多数派”，要么它们中没有谁批准过编号小于n的任何提案，要么它们进行的最近一次批准具有值v。

当且仅当 acceptors 没有收到编号大于n的请求时，acceptors 才批准编号为n的提案。

2.3 分布式锁服务Chubby

• Paxos算法逻辑

每次只批准
一个决议

解决多个“多数派”

p1: 每个acceptor只接受它得到的第一个决议。

解决没有“多数派”

p2: 一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

稍作增强

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

为了解决p2a和p1的矛盾

p2b: 一旦某个决议v得到通过，之后任何proposer再提出的决议必须是v。

解决p2b实现困难的问题

p2c: 如果一个编号为n的提案具有值v，那么存在一个“多数派”，要么它们中没有谁批准过编号小于n的任何提案，要么它们进行的最近一次批准具有值v。

为了保证决议的唯一性

当且仅当 acceptors 没有收到编号大于n的请求时，acceptors 才批准编号为n的提案。

10.1 Paxos算法

10.1.1 Paxos 算法背景知识

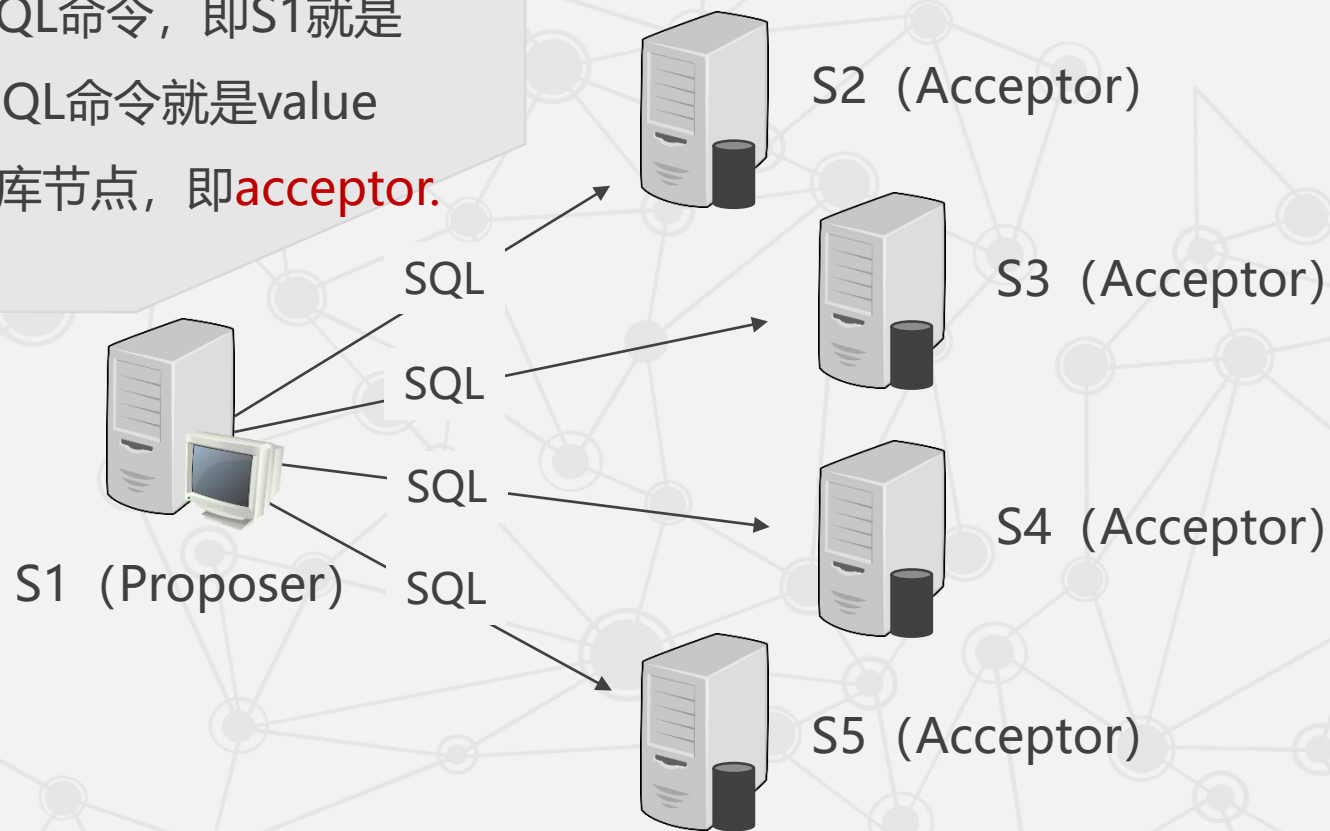
10.1.2 Paxos 算法详解

► 10.1.3 Paxos 算法举例

10.1 Paxos算法

• Paxos 算法举例

- 假设S1只发送SQL命令，即S1就是 **proposer**，而SQL命令就是value
- S2-S5均为数据库节点，即**acceptor**.

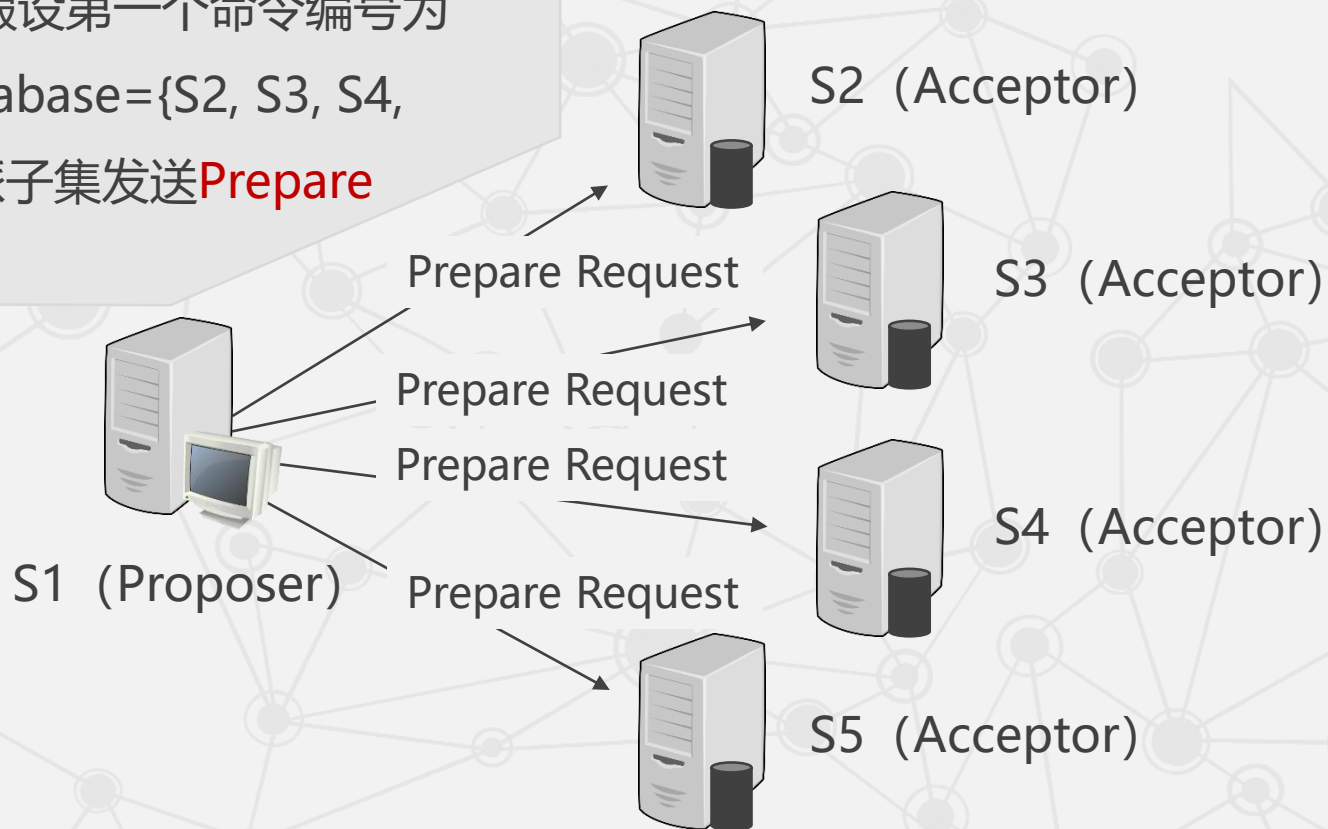


10.1 Paxos算法

• Paxos 算法举例

步骤一

S1选定编号1（假设第一个命令编号为1），向集合database={S2, S3, S4, S5}的一个多数派子集发送**Prepare Request (PR)**。

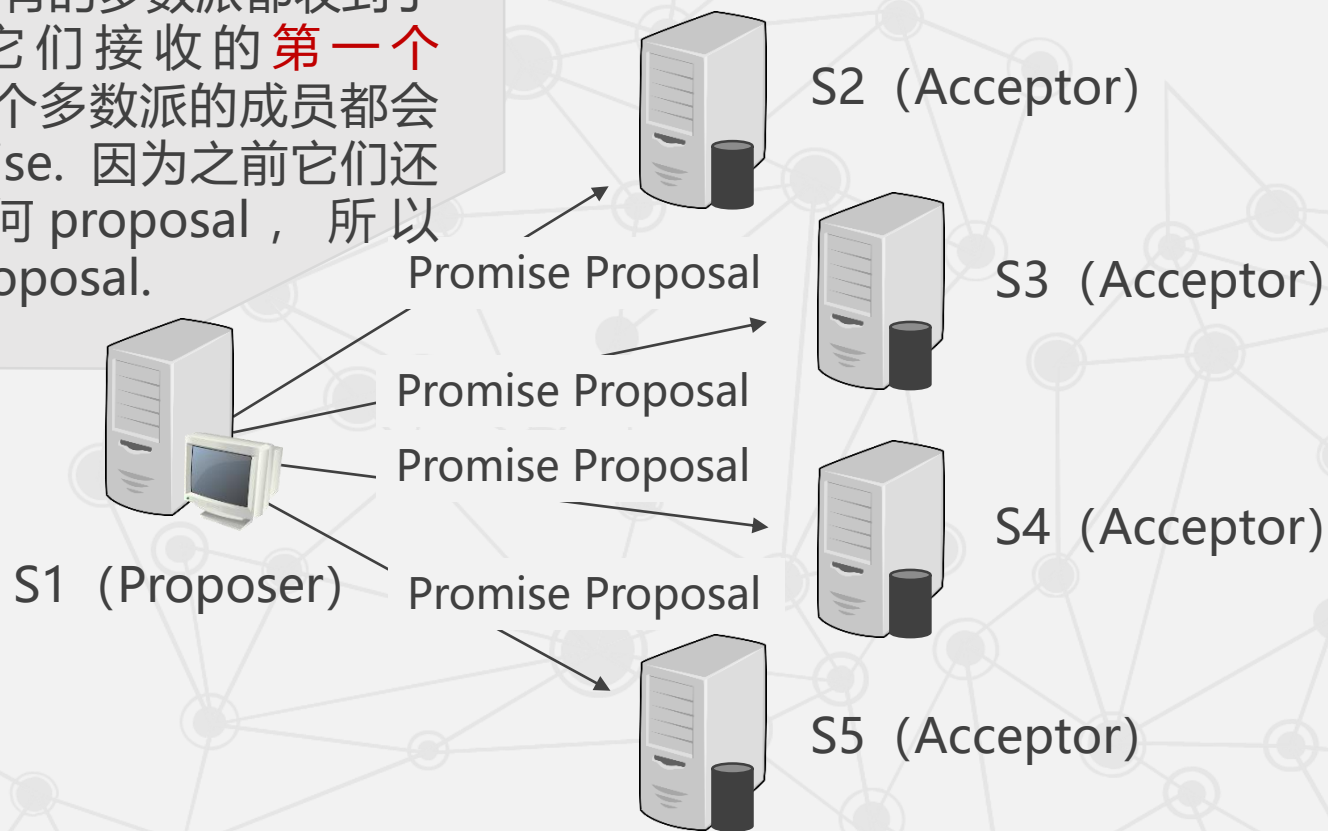


10.1 Paxos算法

• Paxos 算法举例

步骤二

- 如果通信顺利，所有的多数派都收到了PR，因为这是它们接收的**第一个** prepare，所以这个多数派的成员都会回应S1一个promise. 因为之前它们还没有接收过任何 proposal，所以 promise**不带回** proposal.



p1: 每个acceptor只接受它得到的第一个决议。

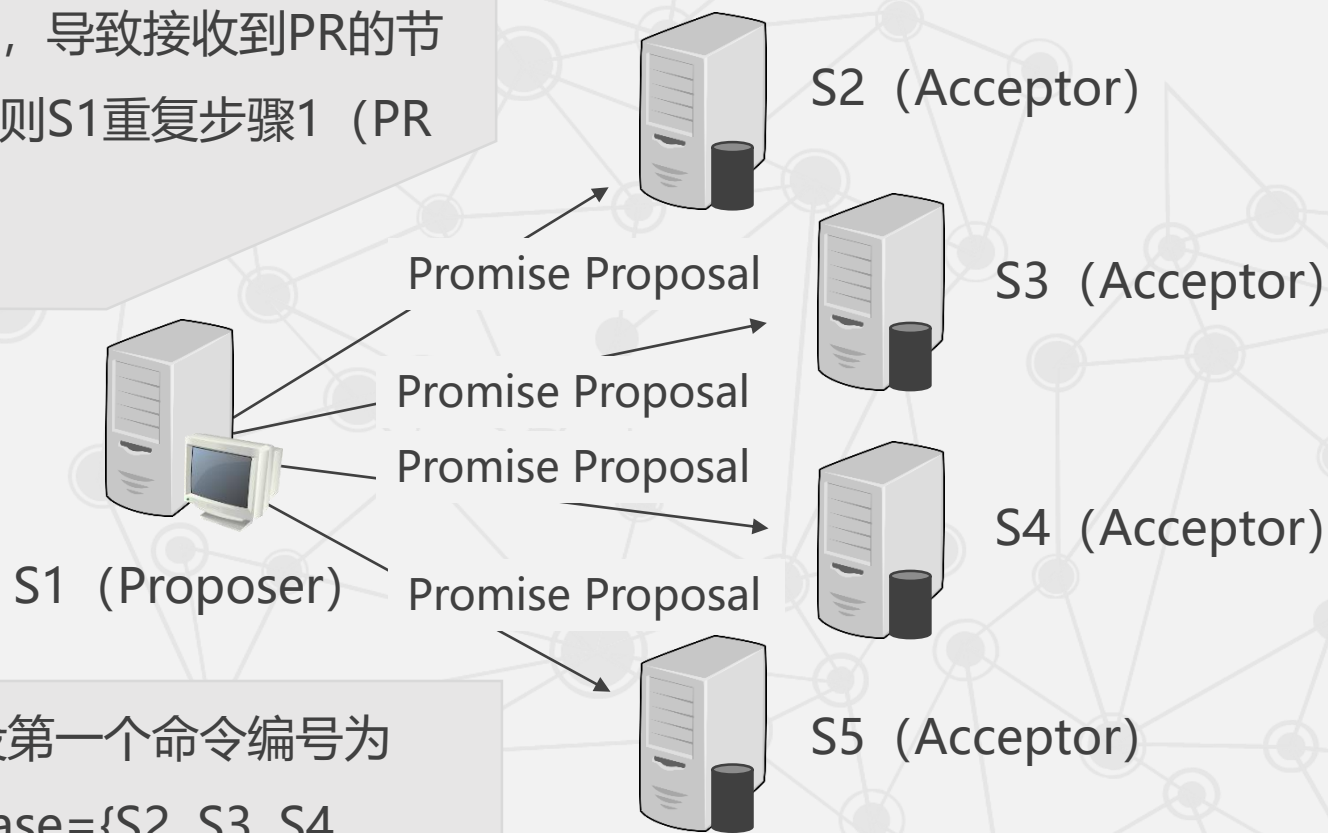
p2c: 如果一个编号为n的提案具有值v，那么存在一个“多数派”，要么它们中没有谁批准过编号小于n的任何提案，要么它们进行的最近一次批准具有值v。

10.1 Paxos算法

• Paxos 算法举例

步骤二

- 如果通信部分失败，导致接收到PR的节点不构成多数派，则S1重复步骤1（PR编号递增）



步骤一

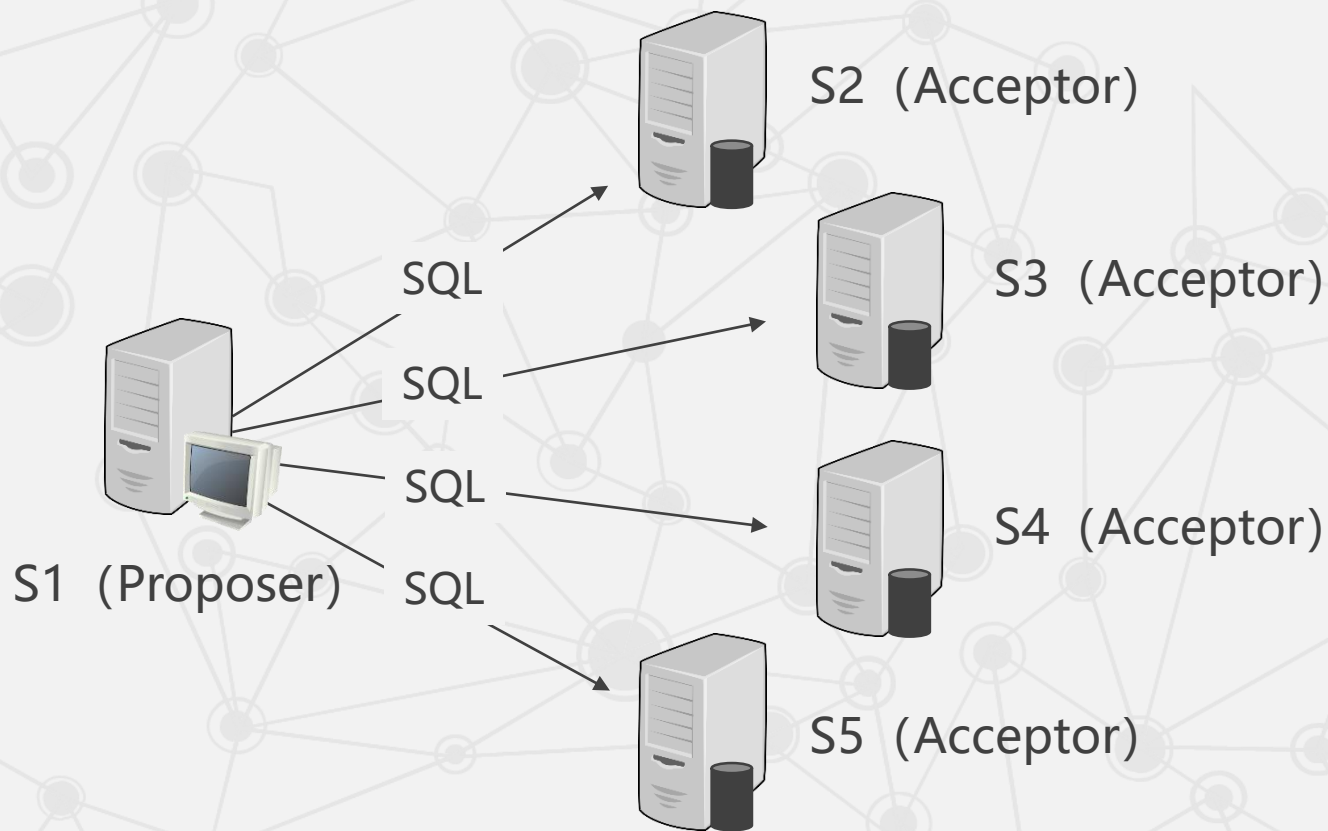
S1选定编号1（假设第一个命令编号为1），向集合database={S2, S3, S4, S5}的一个多数派子集发送Prepare Request (PR)。

10.1 Paxos算法

• Paxos 算法举例

步骤三

S1接收到多数派的**Promise**，向集合database发出带有**第一个 SQL 命令**（这里的SQL命令就是之前的value）的**Proposal**，编号为1，因为Promise没有带回Proposal所以这里的SQL命令没有限制。



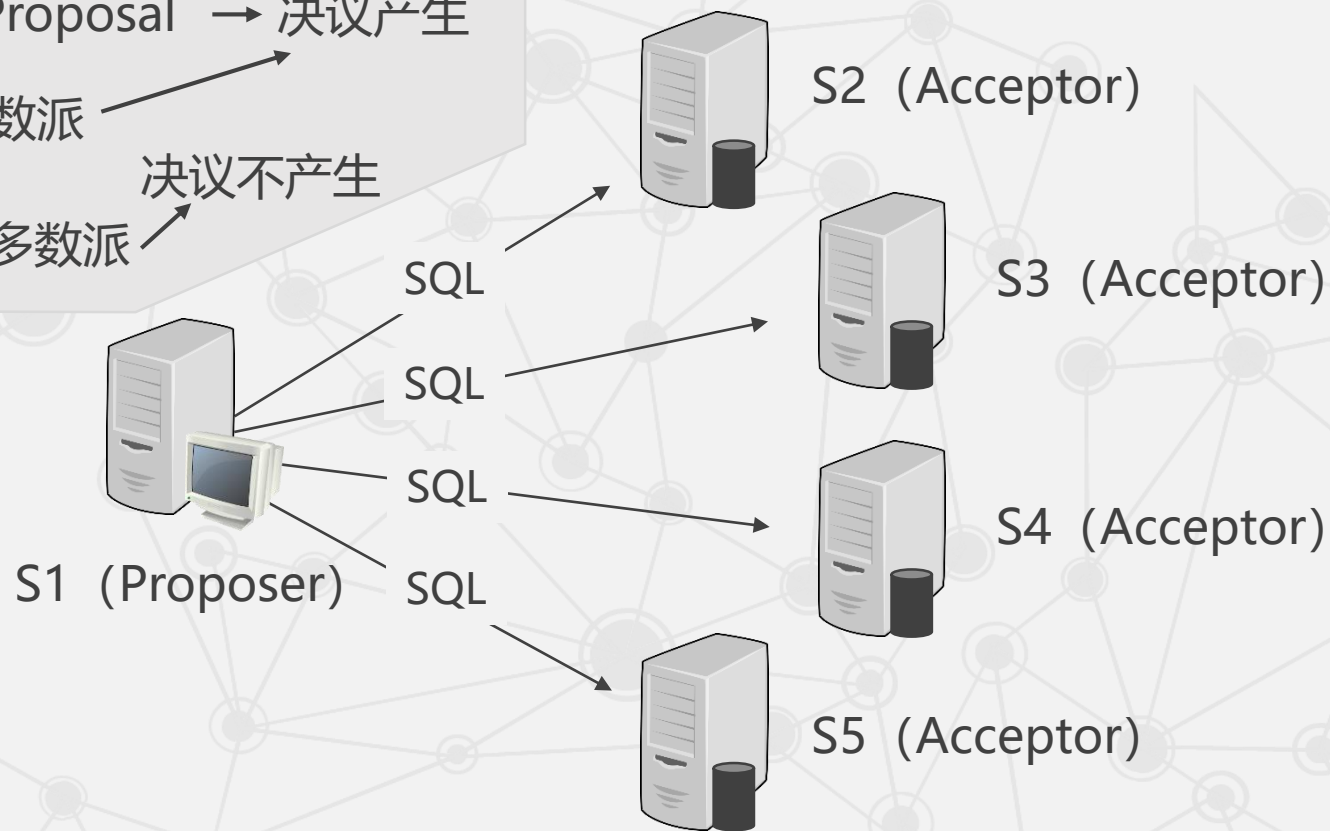
10.1 Paxos算法

• Paxos 算法举例

步骤四

通信顺利 → 接收Proposal → 决议产生

通信失败 → 构成多数派 → 决议产生
通信失败 → 不构成多数派 → 决议不产生

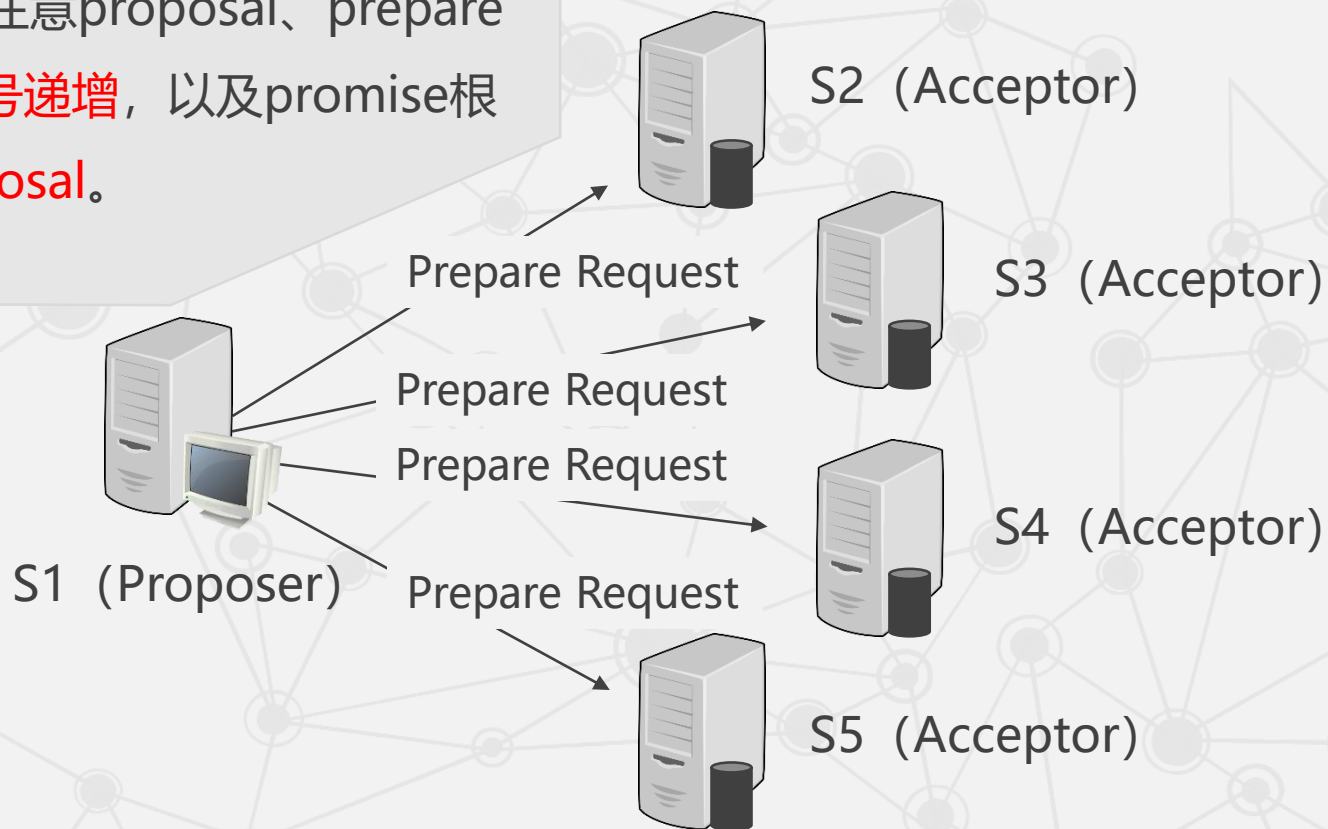


10.1 Paxos算法

• Paxos 算法举例

步骤五

重复以上操作，注意proposal、prepare及promise的**编号递增**，以及promise根据情况**带回proposal**。



2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

一般情况下，以上算法过程就可以成功地解决一致性问题，但是也有特殊情况。

根据算法，一个编号更大的提案会终止之前的提案过程，如果两个 proposer 在这种情况下都转而提出一个编号更大的提案，那么就可能会陷入活锁。

此时，需要选举出一个 president，仅允许 president 提出提案。

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

- 什么是活锁？
- 活锁指的是一个或多个执行者（如线程、进程或任务）没有被阻塞，但由于某些条件未满足，导致它们不断尝试、失败、再尝试的状态。
- 活锁的实体在不断改变状态，通常表现为一种“活”的状态，而没有完全停止。
- 活锁有可能自行解开，因为它依赖于外部条件的改变或资源的释放。

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

- 什么是死锁？
- 死锁是指两个或两个以上的进程在执行过程中，因为**争夺资源**而陷入了一种互相**等待**的状态，无法继续推进。
- 在死锁状态下，每个进程都在**等待**对方释放资源，导致它们都无法获得所需的资源。
- 死锁是一种**僵局**，如果没有外力介入，这些进程将永远无法继续执行。

2.3 分布式锁服务Chubby

• 2.3.1 Paxos算法

- 活锁与死锁的区别？
- 活锁的实体在尝试获取资源，而**没有完全停止**；而死锁的实体则**完全停止**，因为它们在等待对方释放资源。
- 活锁有可能**自行解开**，而死锁则不能，除非有**外部干预**来打破等待状态。

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

► 2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

2.3.4 Chubby文件系统

2.3.5 通信协议

2.3.6 正确性与性能

2.3 分布式锁服务Chubby

• 2.3.2 Chubby系统设计

- 通常情况下，Google的一个数据中心仅运行一个Chubby单元 (Chubby cell) ，
- 这个单元需要支持包括GFS、Bigtable在内的众多Google服务，
- 因此，在设计Chubby的时候，必须充分考虑系统需要实现的目标以及可能出现的各种问题。

2.3 分布式锁服务Chubby

- Chubby的设计目标主要有以下几点

1

高可用性和高可靠性

这是系统设计的首要目标，在保证这一目标的基础上，再考虑系统的吞吐量和存储能力。

2.3 分布式锁服务Chubby

- Chubby的设计目标主要有以下几点

1

高可用性和高可靠性

2

高扩展性

将数据存储在价格较为低廉的RAM，支持大规模用户访问文件。

2.3 分布式锁服务Chubby

- Chubby的设计目标主要有以下几点

1

高可用性和高可靠性

2

高扩展性

3

**支持粗粒度的
建议性锁服务**

提供这种服务的根本目的是提高系统的性能。

2.3 分布式锁服务Chubby

- Chubby的设计目标主要有以下几点

1

高可用性和高可靠性

2

高扩展性

3

支持粗粒度的
建议性锁服务

4

服务信息的直接存储

可以直接存储包括元数据、系统参数在内的有关服务信息，而不需要再维护另一个服务。

2.3 分布式锁服务Chubby

- Chubby的设计目标主要有以下几点

1

高可用性和高可靠性

2

高扩展性

3

支持粗粒度的
建议性锁服务

4

服务信息的直接存储

5

支持通报机制

客户可以及时地了解到事件的发生。

2.3 分布式锁服务Chubby

- Chubby的设计目标主要有以下几点

1

高可用性和高可靠性

2

高扩展性

3

支持粗粒度的
建议性锁服务

4

服务信息的直接存储

5

支持通报机制

6

支持缓存机制

通过一致性缓存将常用信息保存在客户端，避免了频繁地访问主服务器。

2.3 分布式锁服务Chubby

Google没有直接实现一个包含了Paxos算法的函数库，而是设计了全新的锁服务Chubby，原因有三：

1

单独的锁服务可以保证原有系统的架构不变，而函数库不可以。

2

系统中很多事件的发生是需要告知其他用户和服务器的，而基于文件系统的锁服务可以将系统变动写入文件中，从而避免因大量的系统组件之间的通信带来的系统性能的下降。

3

基于锁的开发接口容易被开发者接受。

2.3 分布式锁服务Chubby

1

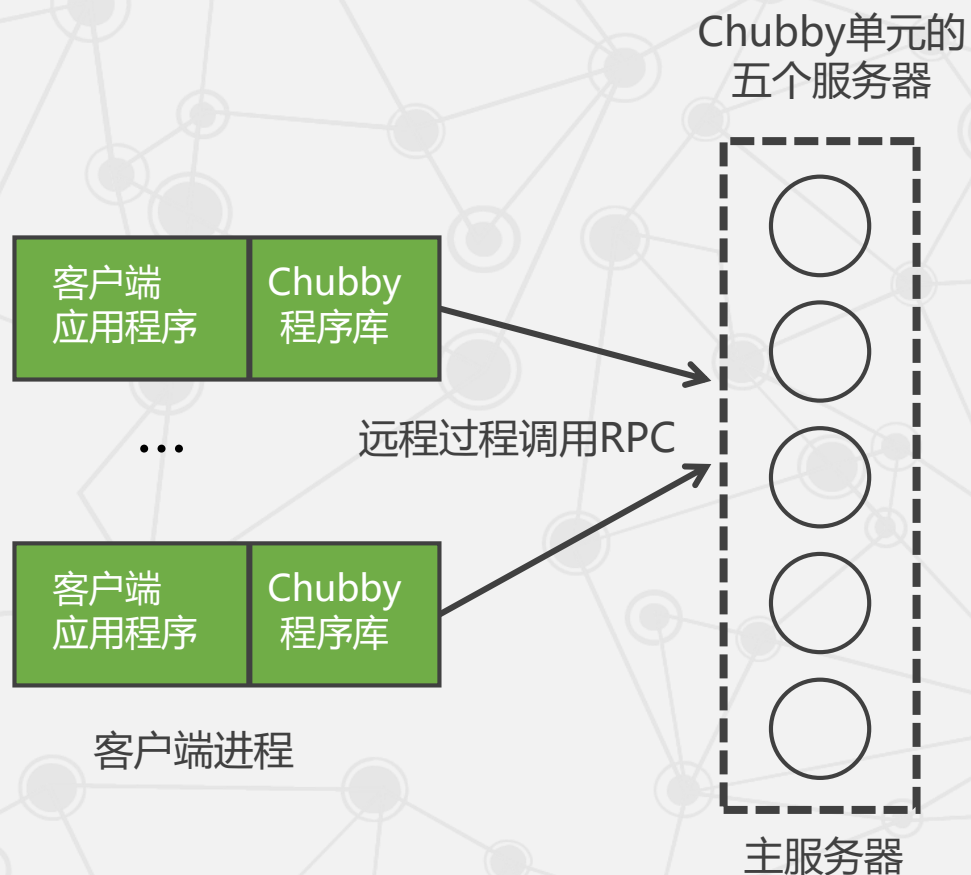
此外，Chubby系统中采用了**建议性**的锁，而非强制性的锁。两者的根本区别在于用户访问某个被锁定的文件时，建议性的锁不会阻止访问，方便了系统组件之间的信息交互。

2

Chubby还采用了**粗粒度**锁服务，而非细粒度锁服务。细粒度的锁持有时间很短，几秒或更少，而粗粒度的锁持有时间可长达几天。粗粒度的锁可以减少频繁换锁带来的系统开销。

2.3 分布式锁服务Chubby

• Chubby的基本架构



客户端

在客户这一端每个客户应用程序都有一个Chubby程序库 (Chubby Library)，客户端的所有应用都是通过调用这个库中的相关函数来完成的。

服务器端

服务器端称为Chubby单元，一般是由五个副本 (Replica) 服务器组成，这五个副本在配置上完全一致，并且在系统刚开始时处于对等地位。

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

► 2.3.3 Chubby中的Paxos

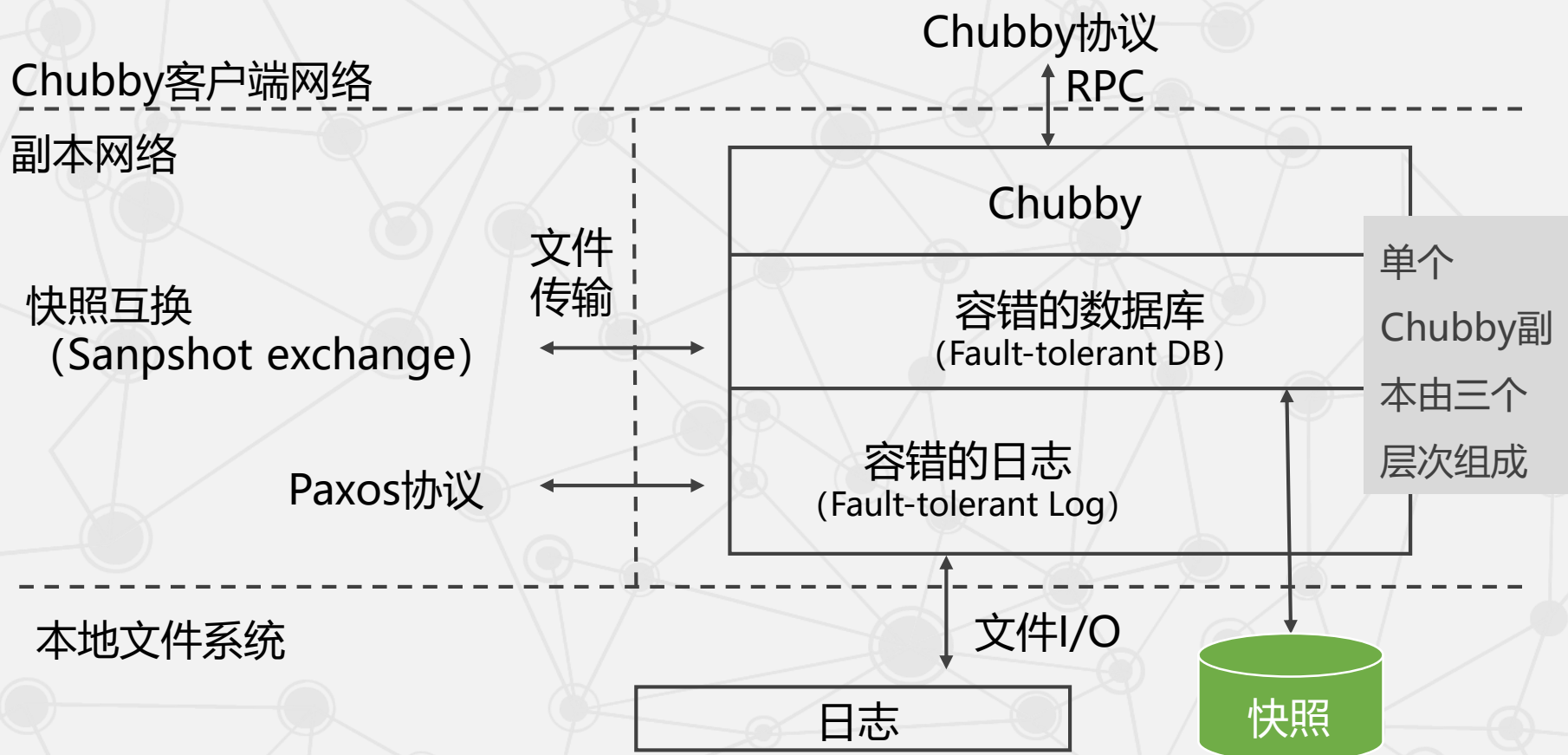
2.3.4 Chubby文件系统

2.3.5 通信协议

2.3.6 正确性与性能

2.3 分布式锁服务Chubby

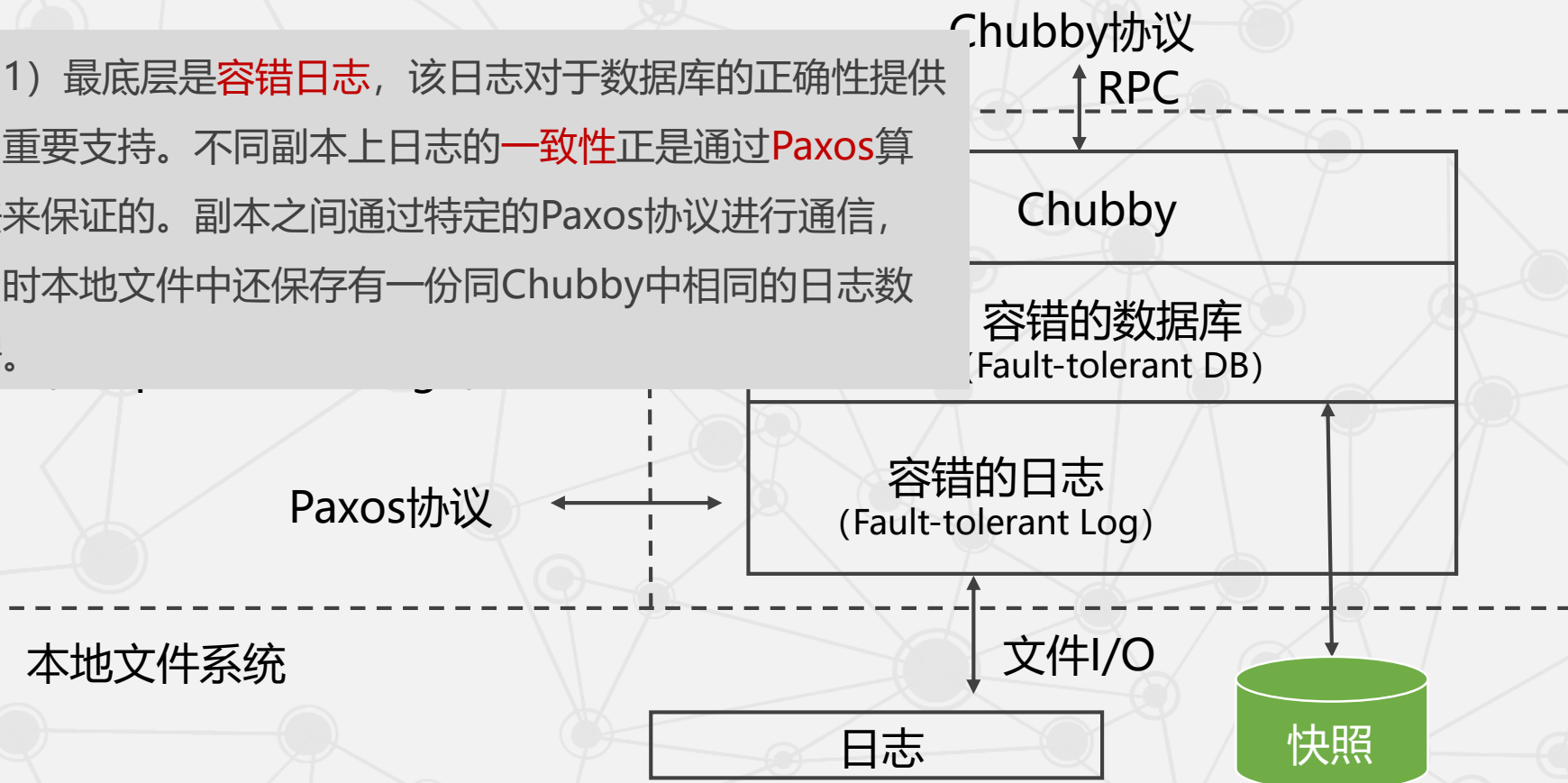
• 单个Chubby副本结构



2.3 分布式锁服务Chubby

• 单个Chubby副本结构

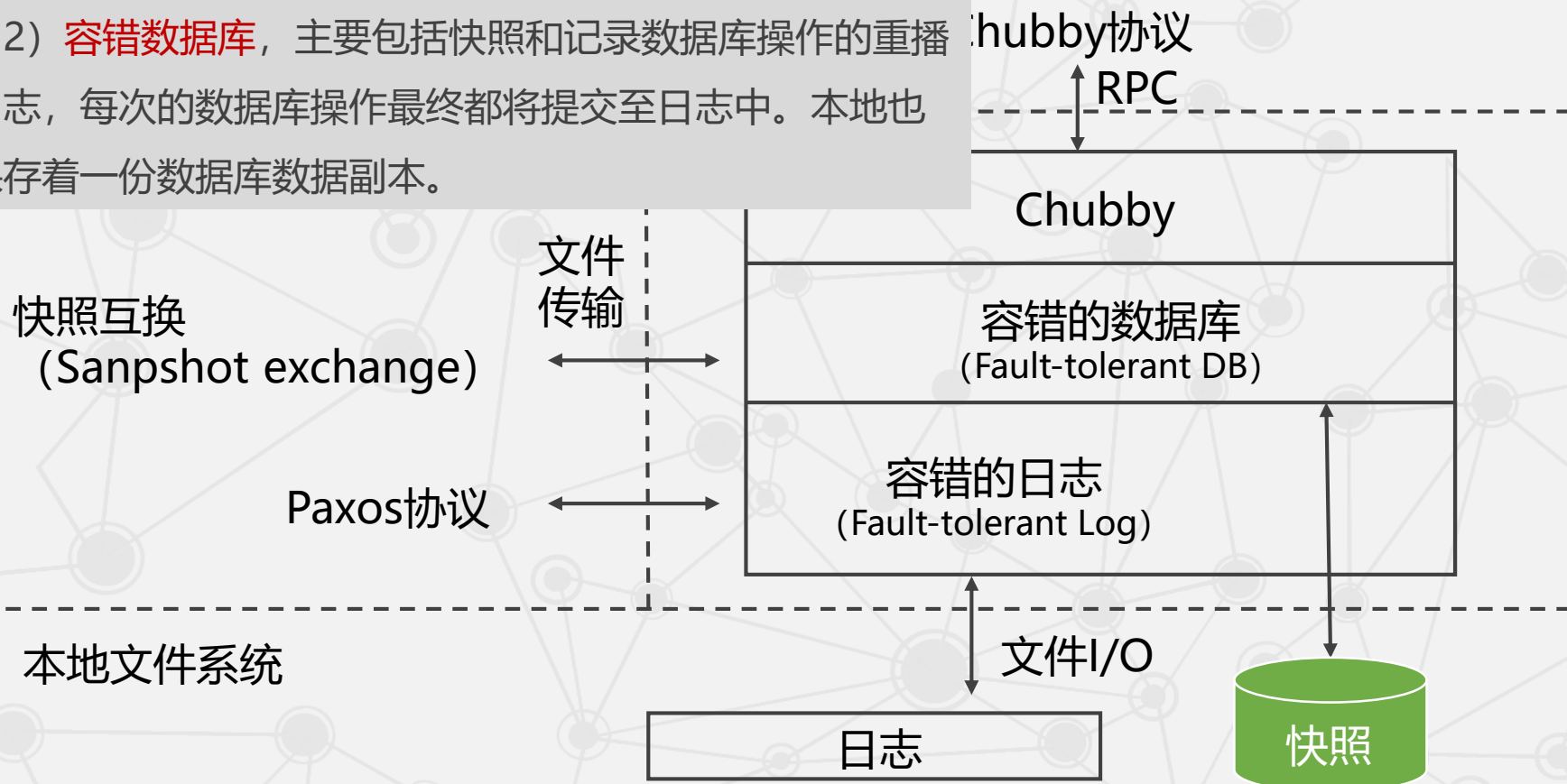
(1) 最底层是**容错日志**，该日志对于数据库的正确性提供了重要支持。不同副本上日志的**一致性**正是通过**Paxos**算法来保证的。副本之间通过特定的Paxos协议进行通信，同时本地文件中还保存有一份同Chubby中相同的日志数据。



2.3 分布式锁服务Chubby

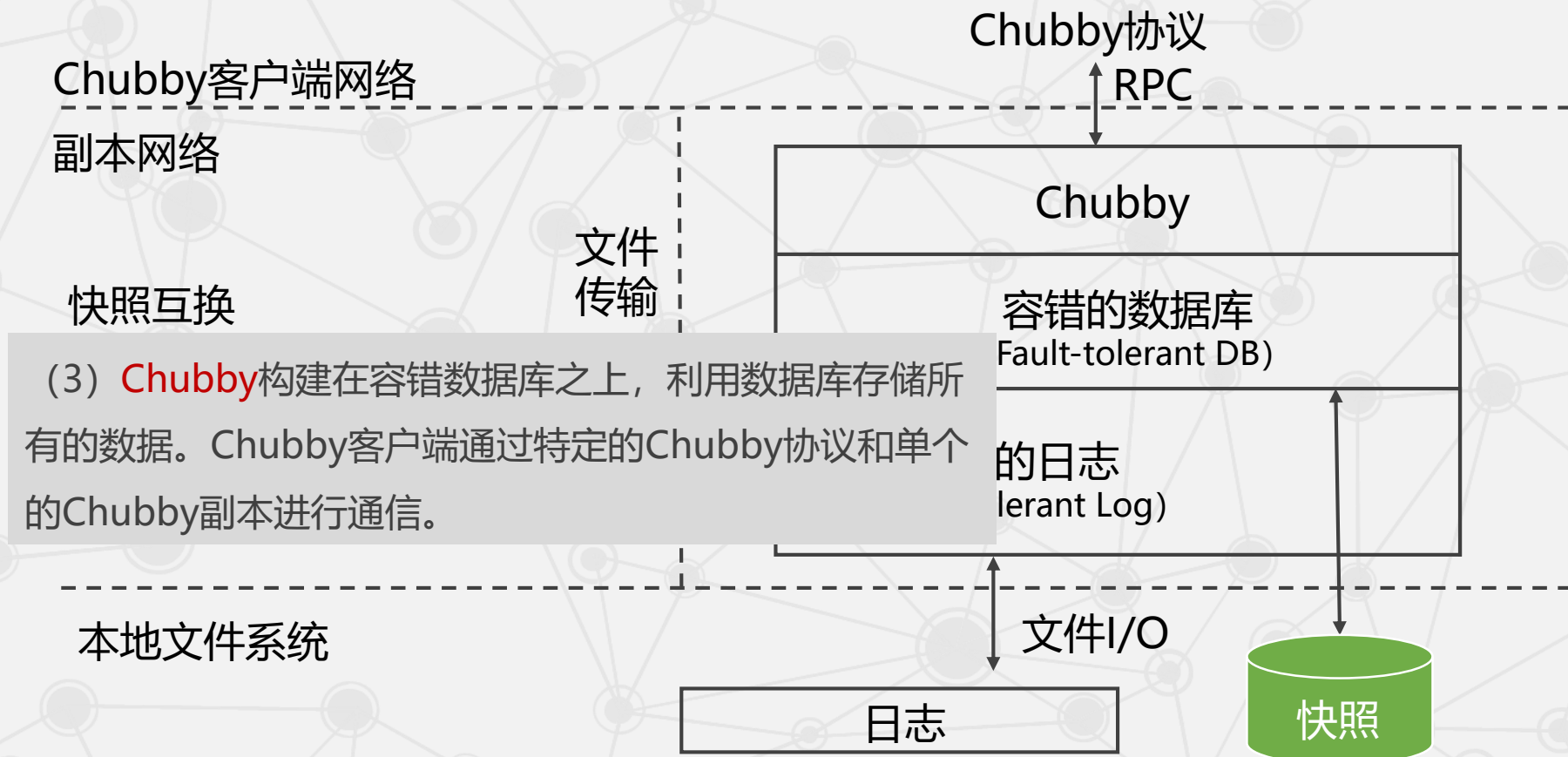
• 单个Chubby副本结构

(2) **容错数据库**，主要包括快照和记录数据库操作的重播日志，每次的数据库操作最终都将提交至日志中。本地也保存着一份数据库数据副本。



2.3 分布式锁服务Chubby

• 单个Chubby副本结构



2.3 分布式锁服务Chubby

• 单个Chubby副本结构

Chubby客户端网络

副本网络

快照互换
(Sanpshot exchange)

Paxos协议

本地文件系统

文件
传输

Chubby

容错的数据库
(Fault-tolerant DB)

容错的日志
(Fault-tolerant Log)

文件I/O

日志

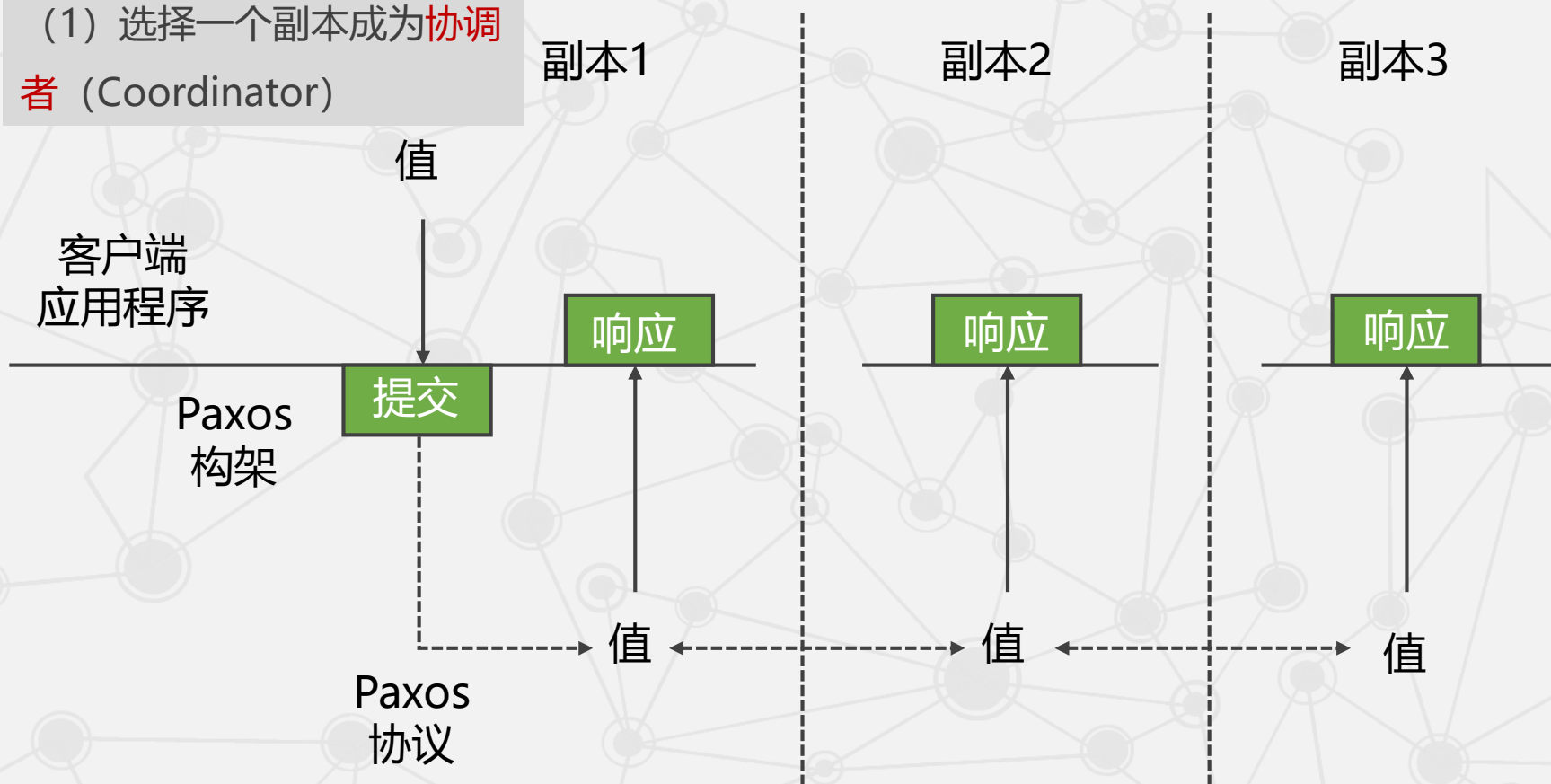
快照

由于副本之间的一致性问题，客户端每次向容错的日志中提交新的值时，Chubby就会自动调用Paxos架构保证不同副本之间数据的一致性。

2.3 分布式锁服务Chubby

● Chubby中的Paxos算法的实际作用过程

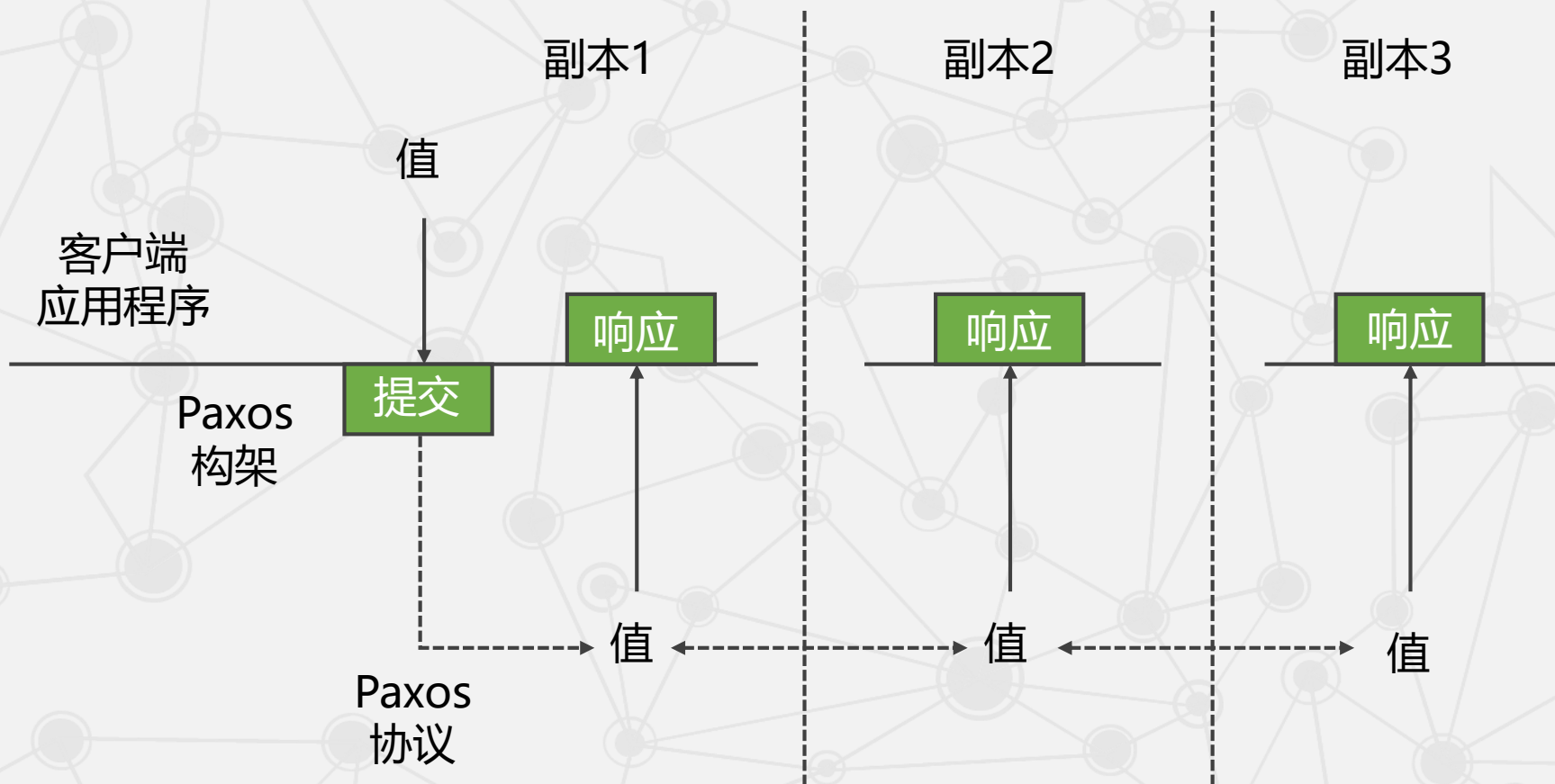
(1) 选择一个副本成为**协调者** (Coordinator)



2.3 分布式锁服务C

• Chubby中的Paxos

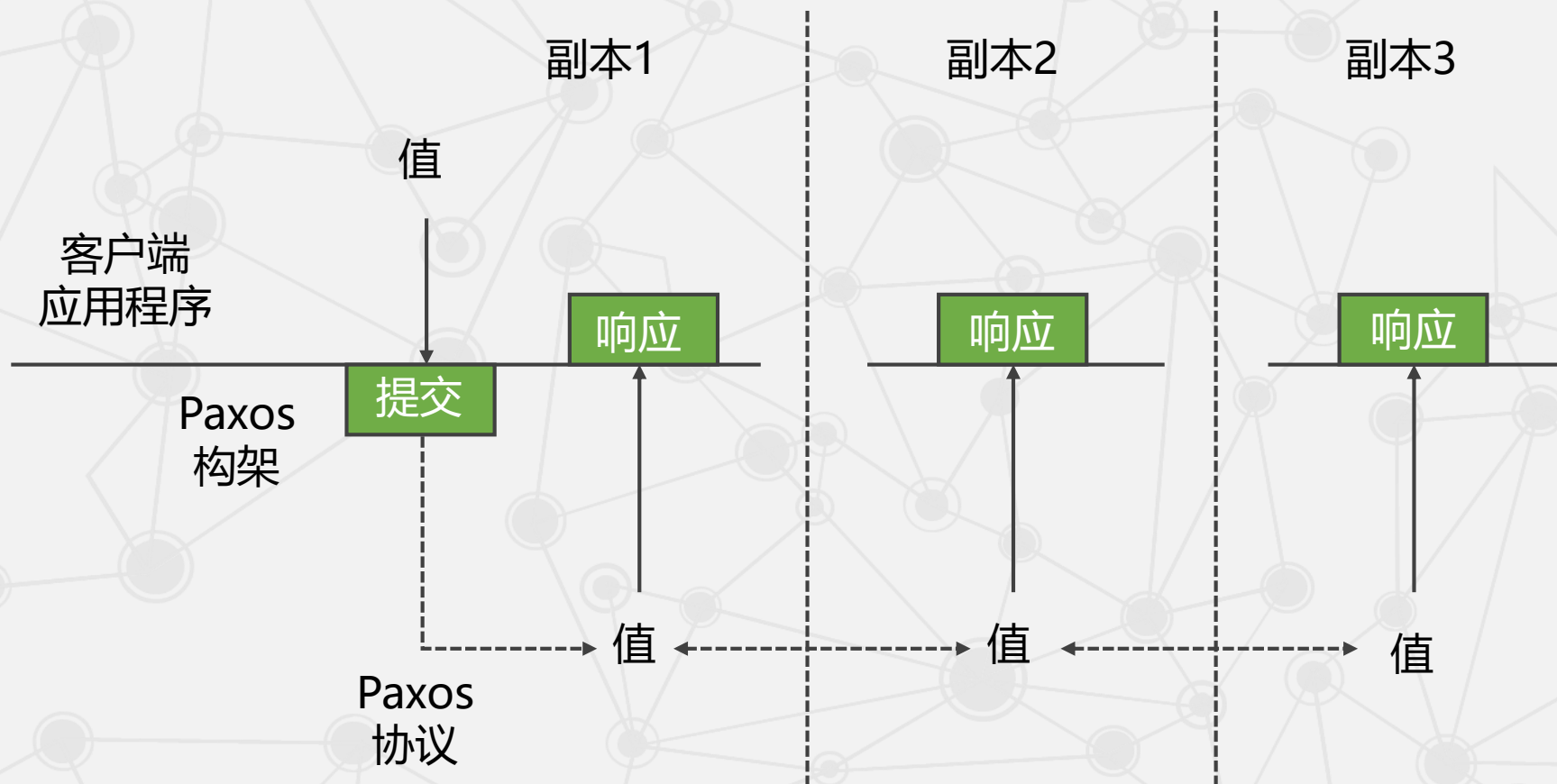
(2) 协调者从客户提交的值中选择一个，然后通过accept消息广播给所有副本，其他的副本可以选择接受或拒绝这个值，并将决定反馈给协调者。



2.3 分布式锁服务C

• Chubby中的Paxos

(3) 一旦协调者收到大多数副本的**接受**消息后，就认为达到了一致性，接着协调者向相关的副本发送一个commit消息。

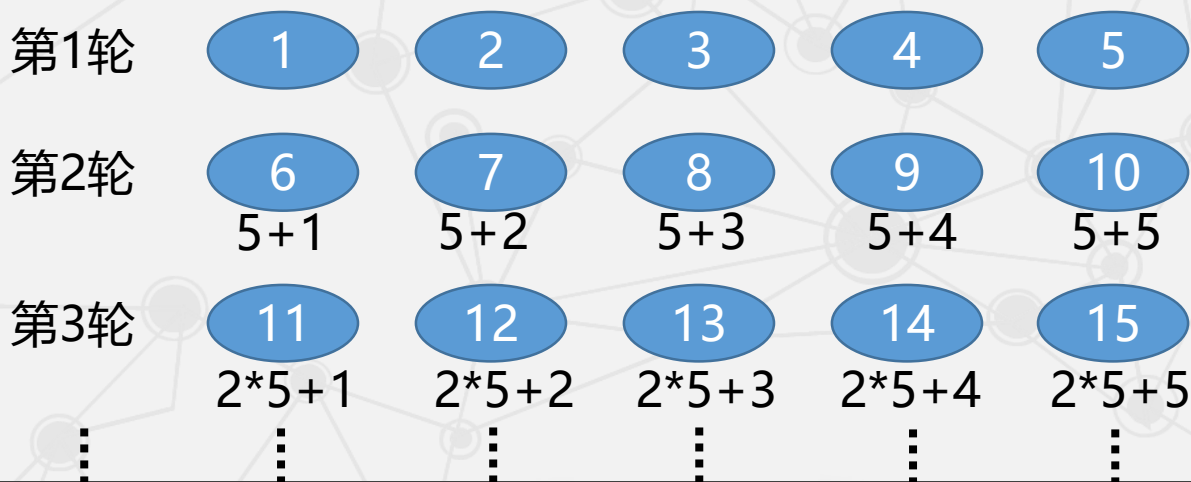


2.3 分布式锁服务Chubby

上述三个过程实际上跟Paxos的核心思想是完全一致的，这些过程保证提交到不同副本上容错日志中数据是完全一致的，进而保证Chubby中数据的一致性。

由于单个的协调者可能失效，系统允许同时有**多个协调者**，但多个协调者可能会导致提交了多个不同的值。对此，Chubby借鉴了Paxos，给协调者指派**序号**或限制协调者可以选择的**值**。Chubby**指派序号**的方法：

(1) 在一个有 n 个副本的系统中，为每个副本分配一个id i_r ，其中 $0 \leq i_r \leq n - 1$ 。则副本的序号 $s = k \times n + i_r$ ，其中 k 的初始值为0。



2.3 分布式锁服务Chubby

上述三个过程实际上跟Paxos的核心思想是完全一致的，这些过程保证提交到不同副本上容错日志中数据是完全一致的，进而保证Chubby中数据的一致性。

由于单个的协调者可能失效，系统允许同时有**多个协调者**，但多个协调者可能会导致提交了多个不同的值。对此，Chubby借鉴了Paxos，给协调者指派**序号**或限制协调者可以选择的**值**。Chubby**指派序号**的方法：

(1) 在一个有 n 个副本的系统中，为每个副本分配一个id i_r ，其中 $0 \leq i_r \leq n - 1$ 。则副本的序号 $s = k \times n + i_r$ ，其中 k 的初始值为0。

(2) 某个副本想成为协调者时，它就根据规则生成一个比它以前的序号更大的序号（实际上就是提高 k 的值），并将这个序号通过propose消息**广播**给其他所有的副本。

(3) 如果接收到广播的副本发现该序号**比它以前见过的序号都大**，则向发出广播的副本返回一个promise消息，并且承诺不再接受旧的协调者发送的消息。如果大多数副本都返回了promise消息，则新的协调者就产生了。

当且仅当 acceptors 没有收到编号大于 n 的请求时，acceptors 才批准编号为 n 的提案。

2.3 分布式锁服务Chubby

针对“限制协调者可以选择的值”这一解决方法，Paxos强制新的协调者必须选择**和前任相同的值**。

p2b：一旦某个决议v得到通过，之后任何proposer再提出的决议必须是v。

为了提高系统的效率，Chubby做了一个重要的优化，那就是在选择某一个副本作为协调者之后就**长期不变**，此时，协调者就被称为**主服务器**（Master）。产生一个主服务器避免了同时有多个协调者而带来的一些问题。

在Chubby中，客户端的数据请求都是由**主服务器**来完成，Chubby保证在一定的时间内，**有且仅有**一个主服务器，这个时间就称为主服务器的**租约期**（Master Lease）。

如果某个服务器被连续推举为主服务器的话，这个租约期就会不断被更新。租约期内所有的客户请求都由主服务器处理。

客户端如果需要确定主服务器的位置，可以向**DNS**发送一个主服务器**定位**请求，非主服务器的副本将对该请求作出回应。通过这种方式，客户端能够快速、准确地对主服务器进行定位。

2.3 分布式锁服务Chubby

Chubby对Paxos的一些技术细节进行了补充，是基于Paxos实现的，但其技术手段更加丰富、更具实践性。但这也导致了最终实现的Chubby不是一个完全经过理论验证的系统。

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

► 2.3.4 Chubby文件系统

2.3.5 通信协议

2.3.6 正确性与性能

2.3 分布式锁服务Chubby

Chubby系统本质上就是一个分布式的、存储大量小文件的文件系统，它所有的操作都是在文件的基础上完成的。

Google没有直接实现一个包含了Paxos算法的函数库，而是设计了全新的锁服务Chubby，原因有三：

1

单独的锁服务可以保证原有系统的架构不变，而函数库不可以。

2

系统中很多事件的发生是需要告知其他用户和服务器的，而基于文件系统的锁服务可以将系统变动写入文件中，从而避免因大量的系统组件之间的通信带来的系统性能的下降。

3

基于锁的开发接口容易被开发者接受。

2.3 分布式锁服务Chubby

Chubby系统本质上就是一个分布式的、存储大量小文件的文件系统，它所有的操作都是在文件的基础上完成的。

例如，在Chubby最常用的锁服务中，每一个文件就代表了一个锁，用户通过打开、关闭和读取文件，获取共享锁或独占锁。

选举主服务器的过程中，符合条件的服务器都同时申请打开某个文件，并请求锁住该文件。成功获得锁的服务器自动成为主服务器，并将其地址写入这个文件夹，以便其他服务器和用户可以获知主服务器的地址信息。

2.3 分布式锁服务Chubby

Chubby的文件系统和UNIX类似。例如：

`/ls/foo/wombat/pouch`

ls：代表lock service，这是所有Chubby文件系统的共有前缀；

foo：是某个单元的名称；

`/wombat/pouch`是foo这个单元上的文件目录或者文件名。

由于Chubby自身的特殊服务要求，Google对Chubby做了一些与UNIX不同的改变。

例如，Chubby不支持内部文件的移动；

不记录文件的最后访问时间；

没有符号连接（软连接），类似于Windows系统中的快捷方式；

没有硬连接，类似于别名。

2.3 分布式锁服务Chubby

在具体实现时，Chubby的文件系统由许多节点组成，分为永久型和临时型，每个节点就是一个文件或目录。

节点中保存着包括ACL（Access Control List，访问控制列表）在内的多种系统元数据。

为了用户能够及时了解元数据的变动，系统规定每个节点的元数据都应当包含四种单调递增的64位编号。

2.3 分布式锁服务Chubby

- 单调递增的64位编号

1 实例号

Instance Number

新节点实例号必定大于旧节点的实例号。

2 内容生成号

Content Generation Number

文件内容修改时该号增加。

3 锁生成号

Lock Generation Number

锁被用户持有时该号增加。

4 ACL生成号

ACL Generation Number

ACL名被覆写时该号增加。

2.3 分布式锁服务Chubby

用户在打开某个**节点**（文件或目录）的同时，会获取一个类似于UNIX中文件描述符的**句柄**。

在文件操作中，用户可以将句柄看做一个指向文件系统的**指针**，这个指针支持一系列的操作。

这个**句柄**由以下三个部分组成：

- (1) **校验数位** (Check Digit)：防止其他用户创建或猜测这个句柄。
- (2) **序号** (Sequence Number)：用来确定句柄是由当前还是以前的主服务器创建的。
- (3) **模式信息** (Mode Information)：用于新的主服务器重新创建一个旧的句柄。

2.3 分布式锁服务Chubby

在实际的执行中，为了避免所有的通信都使用序号带来的系统开销增长，Chubby引入了sequencer的概念。

sequencer实际上就是一个序号，只能由锁的持有者在获取锁时，向系统发出请求来获得。

这样一来，Chubby系统中只有涉及锁的操作才需要序号，其他一概不用。

2.3 分布式锁服务Chubby

● 常用的句柄函数及作用

函数名称	作用
Open()	打开某个文件或者目录来创建句柄
Close()	关闭打开的句柄，后续的任何操作都将中止
Poison()	中止当前未完成及后续的操作，但不关闭句柄
GetContentsAndStat()	返回文件内容及元数据
GetStat()	只返回文件元数据
ReadDir()	返回子目录名称及其元数据
SetContents()	向文件中写入内容
SetACL()	设置ACL名称
Delete()	如果该节点没有子节点的话则执行删除操作
Acquire()	获取锁
Release()	释放锁
GetSequencer()	返回一个sequencer
SetSequencer()	将sequencer和某个句柄进行关联
CheckSequencer()	检查某个sequencer是否有效

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

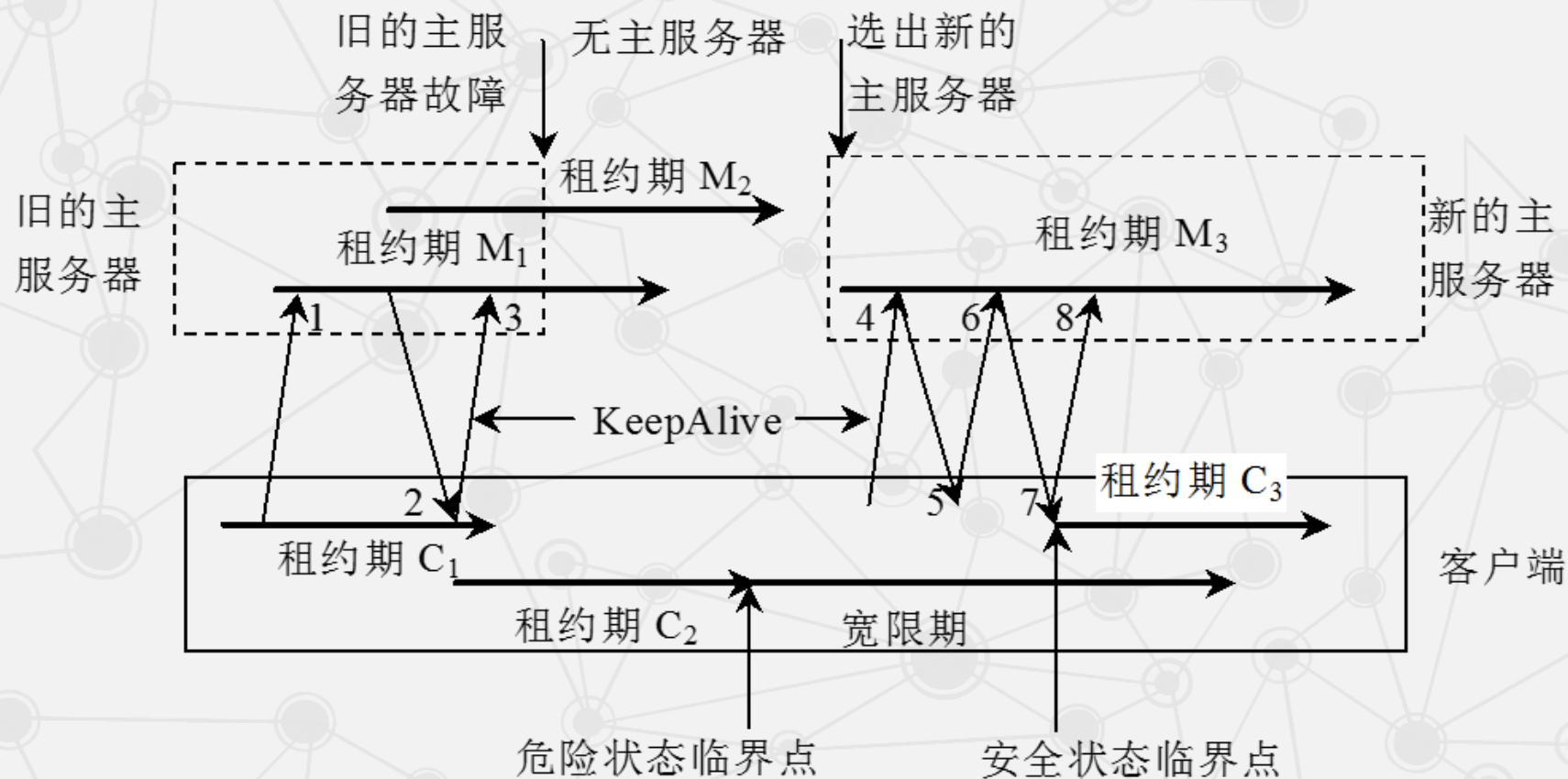
2.3.4 Chubby文件系统

► 2.3.5 通信协议

2.3.6 正确性与性能

2.3 分布式锁服务Chubby

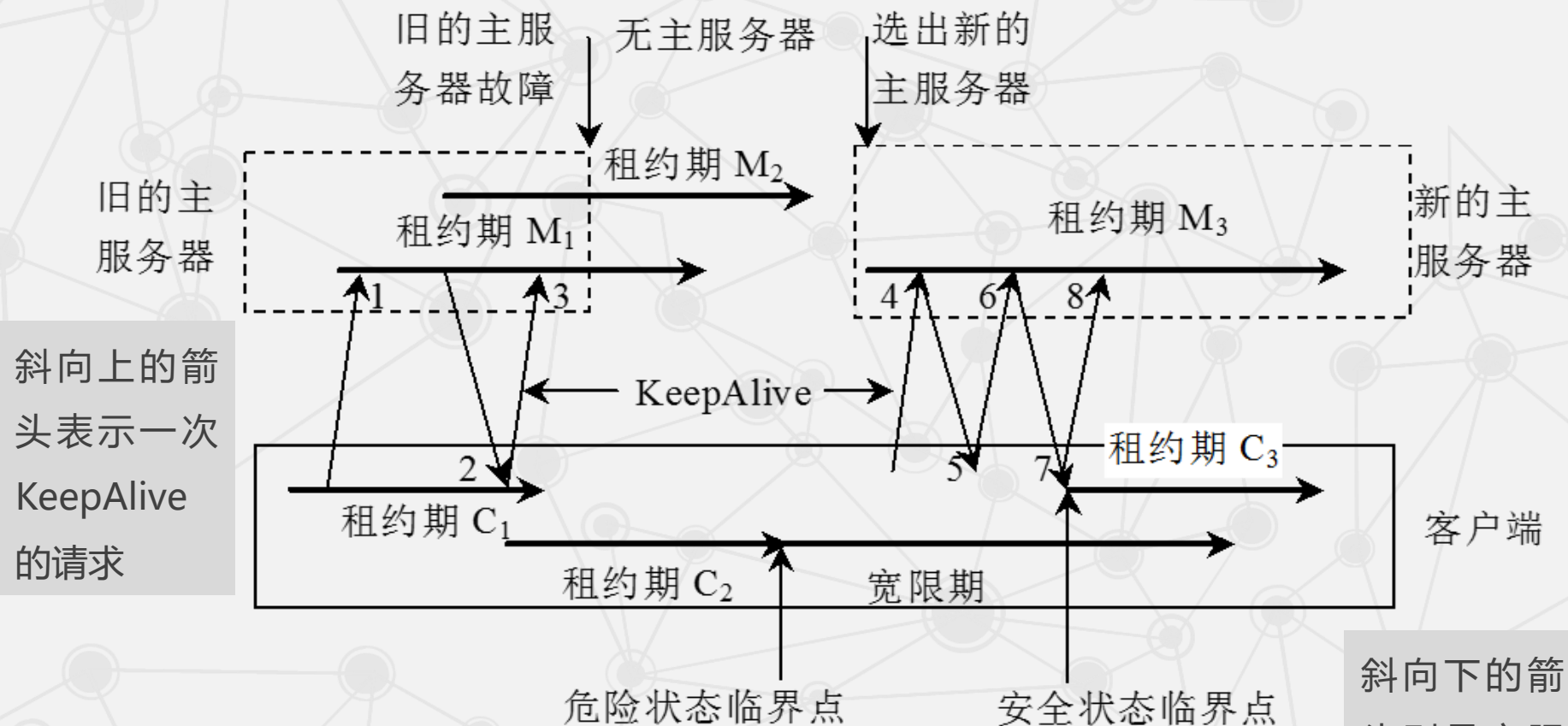
• Chubby客户端与服务器端的通信过程



客户端和主服务器之间的通信是通过KeepAlive握手协议来维持。

2.3 分布式锁服务Chubby

• Chubby客户端与服务器端的通信过程



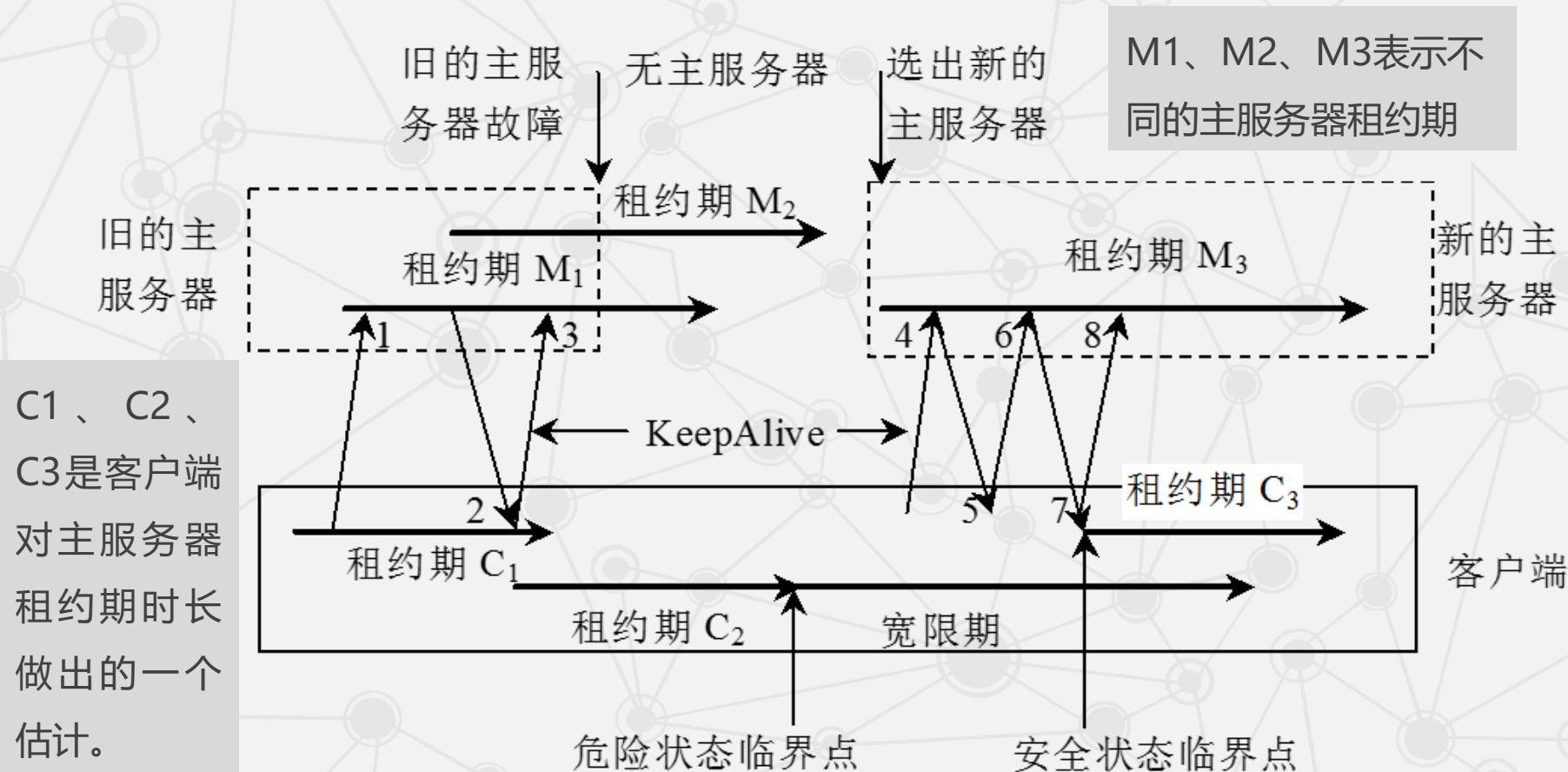
斜向上的箭头表示一次KeepAlive的请求

斜向下的箭头则是主服务器的一次回应。

从左到右的水平方向表示时间在增加。

2.3 分布式锁服务Chubby

• Chubby客户端与服务器端的通信过程



2.3 分布式锁服务Chubby

KeepAlive是周期性发送的一种信息，主要功能是[延迟租约的有效期](#)和[携带事件信息](#)告诉用户更新。

主要的事件包括：

- ◆ 文件内容被修改
- ◆ 子节点的增加、删除和修改
- ◆ 主服务器出错
- ◆ 句柄失效

2.3 分布式锁服务Chubby

- 可能出现的两种故障

系统存在一定的故障概率，常见的有：



1

客户端租约过期

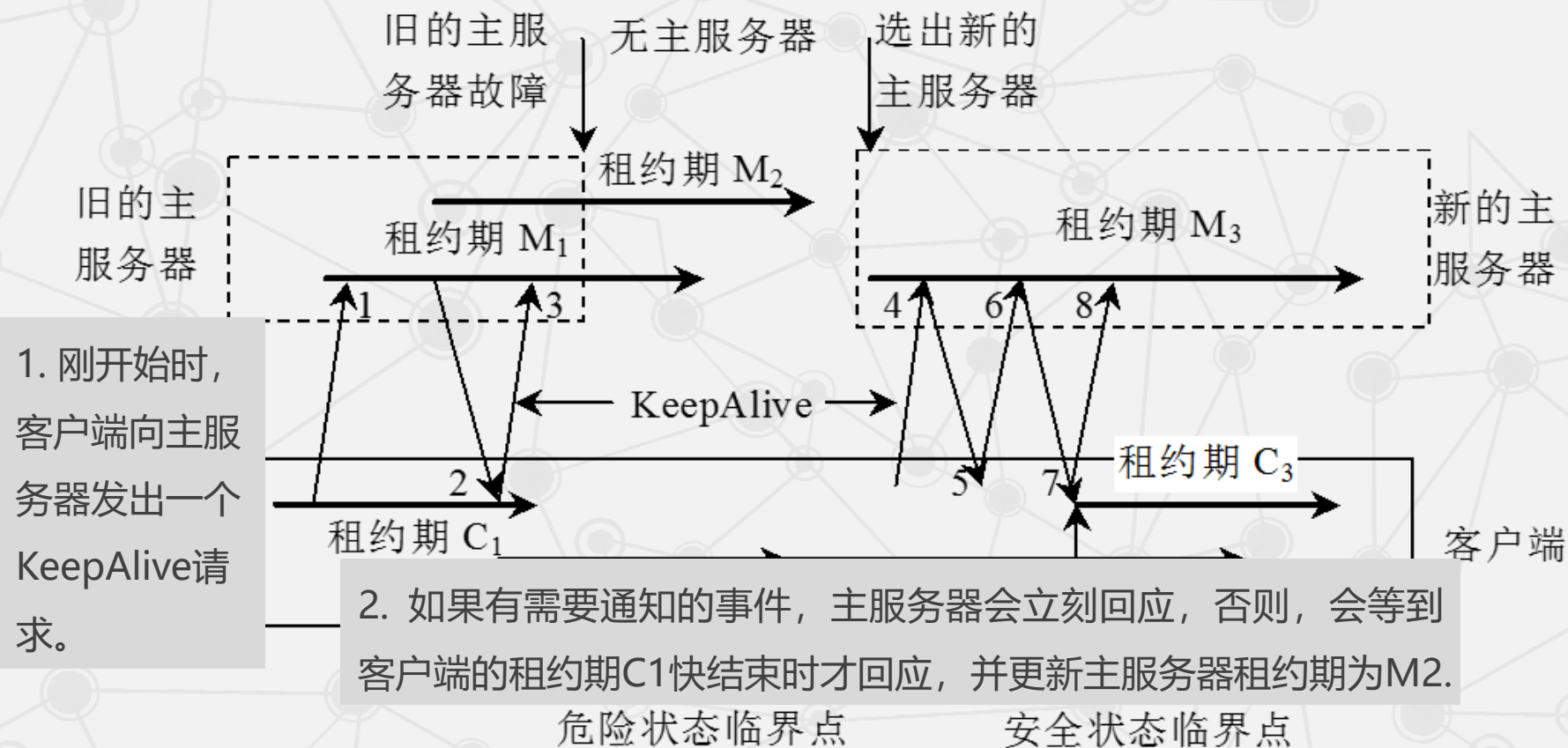


2

主服务器出错

2.3 分布式锁服务Chubby

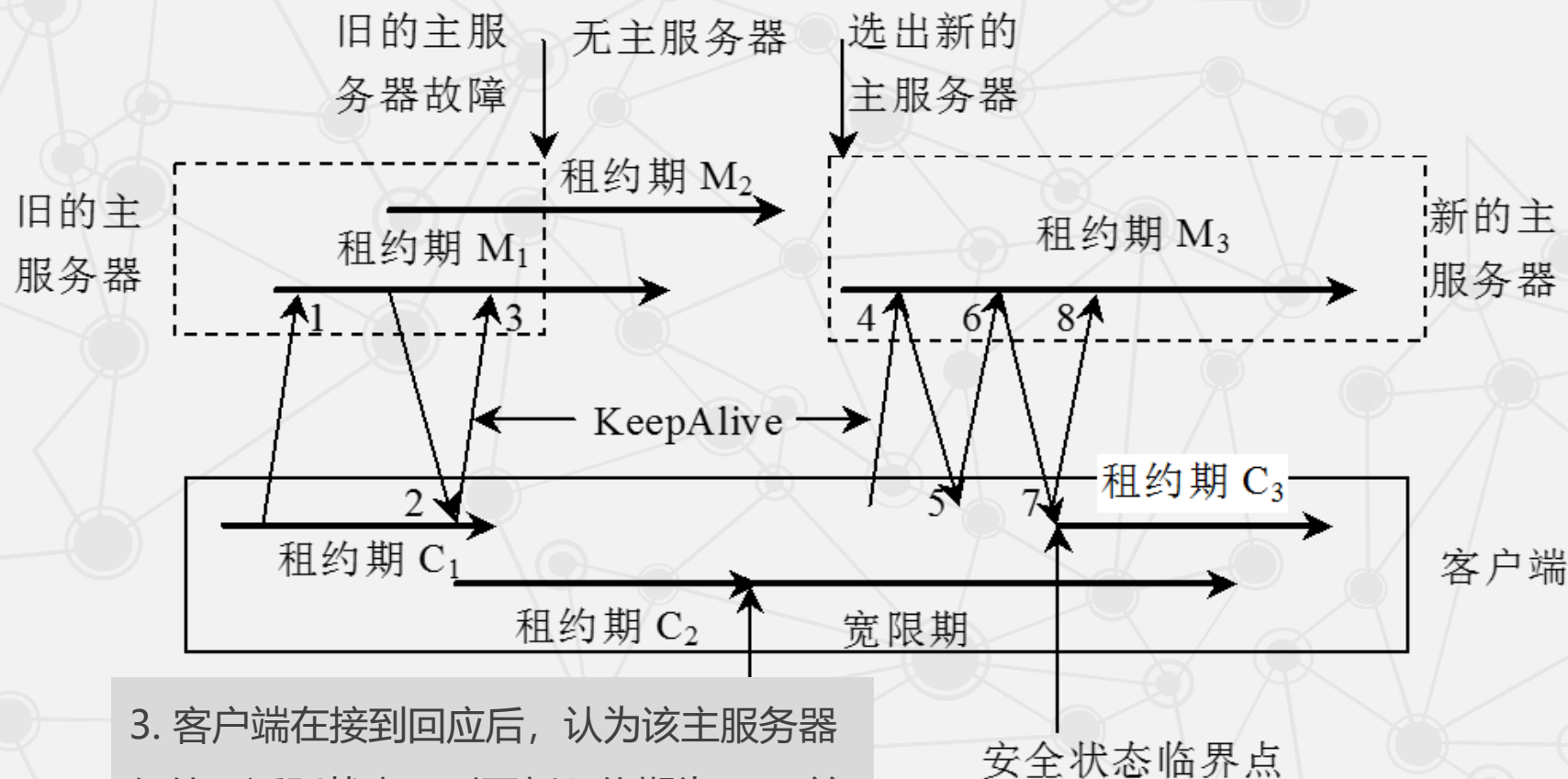
• 1. 客户端租约过期



2.3 分布式锁服务Chubby

● 1. 客户

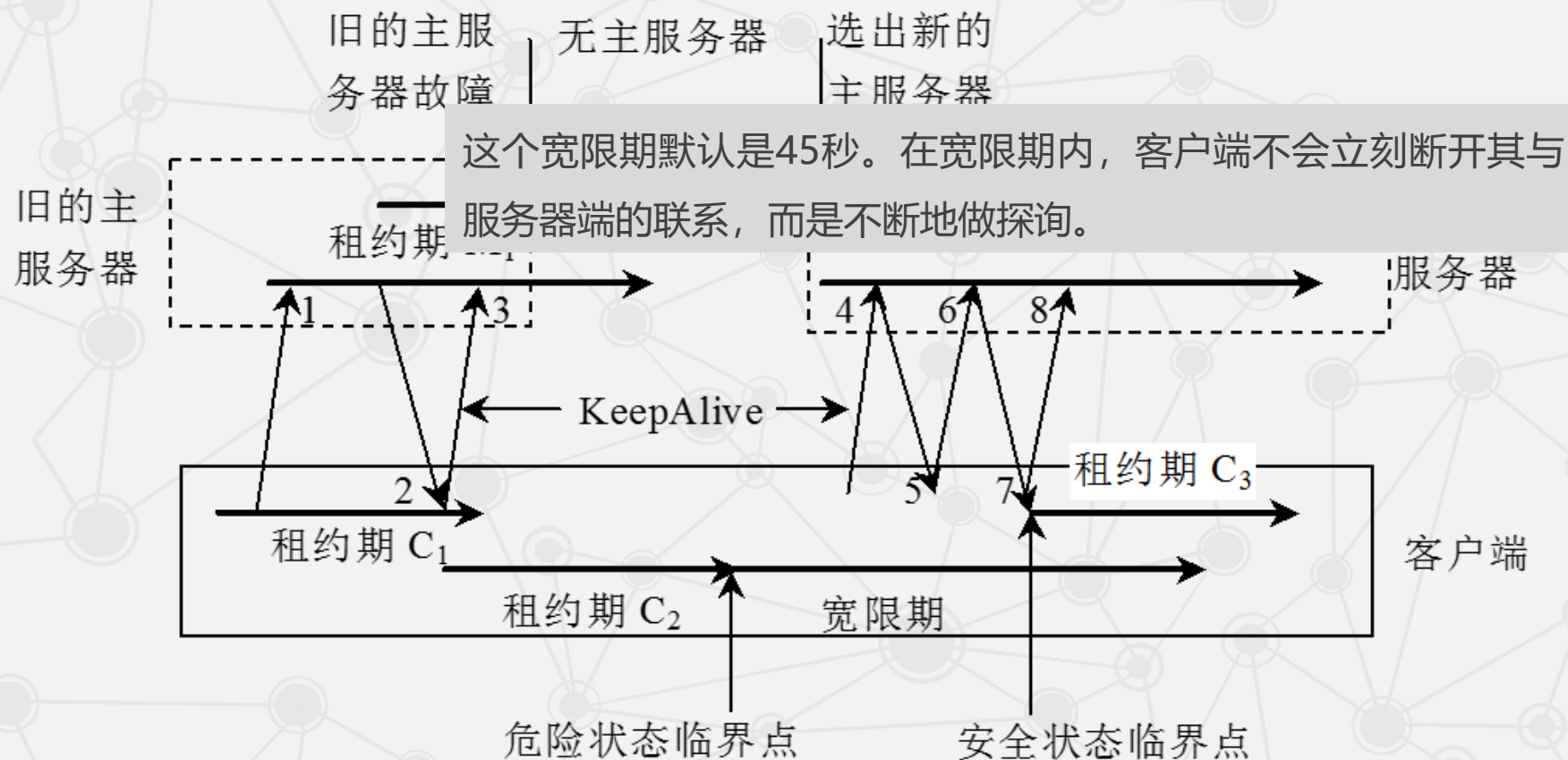
同样的，主服务器可能不是立刻回应，而是等待C2接近结束，但在这个过程中主服务器出现故障停止使用。



3. 客户端在接到回应后，认为该主服务器仍处于活跃状态，则更新租约期为C2，并立刻发出新的KeepAlive请求。

2.3 分布式锁服务Chubby

• 1. 客户端租约过期

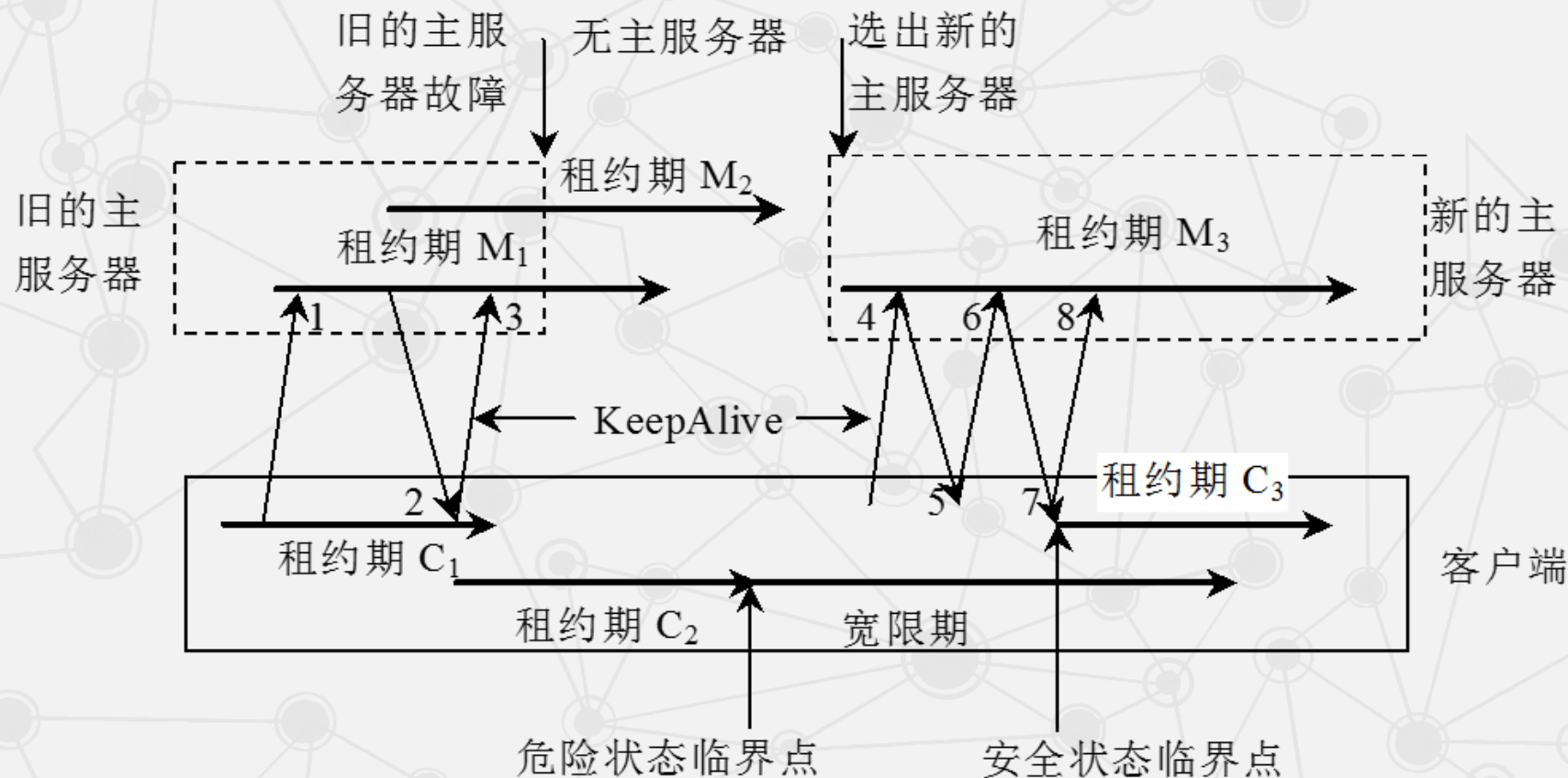


在等待一段时间后，C2到期，由于没收到主服务器的回应，系统向客户端发出一个**危险事件**，客户端清空并暂时停用自己的缓存，从而进入一个**宽限期**的**危险状态**。

2.3 分布式锁服务Chubby

● 1. 客户端租约过期

不同主服务器的**纪元号**不相同，客户端的每次请求都需要这个号来保证处理的请求是针对当前的主服务器。

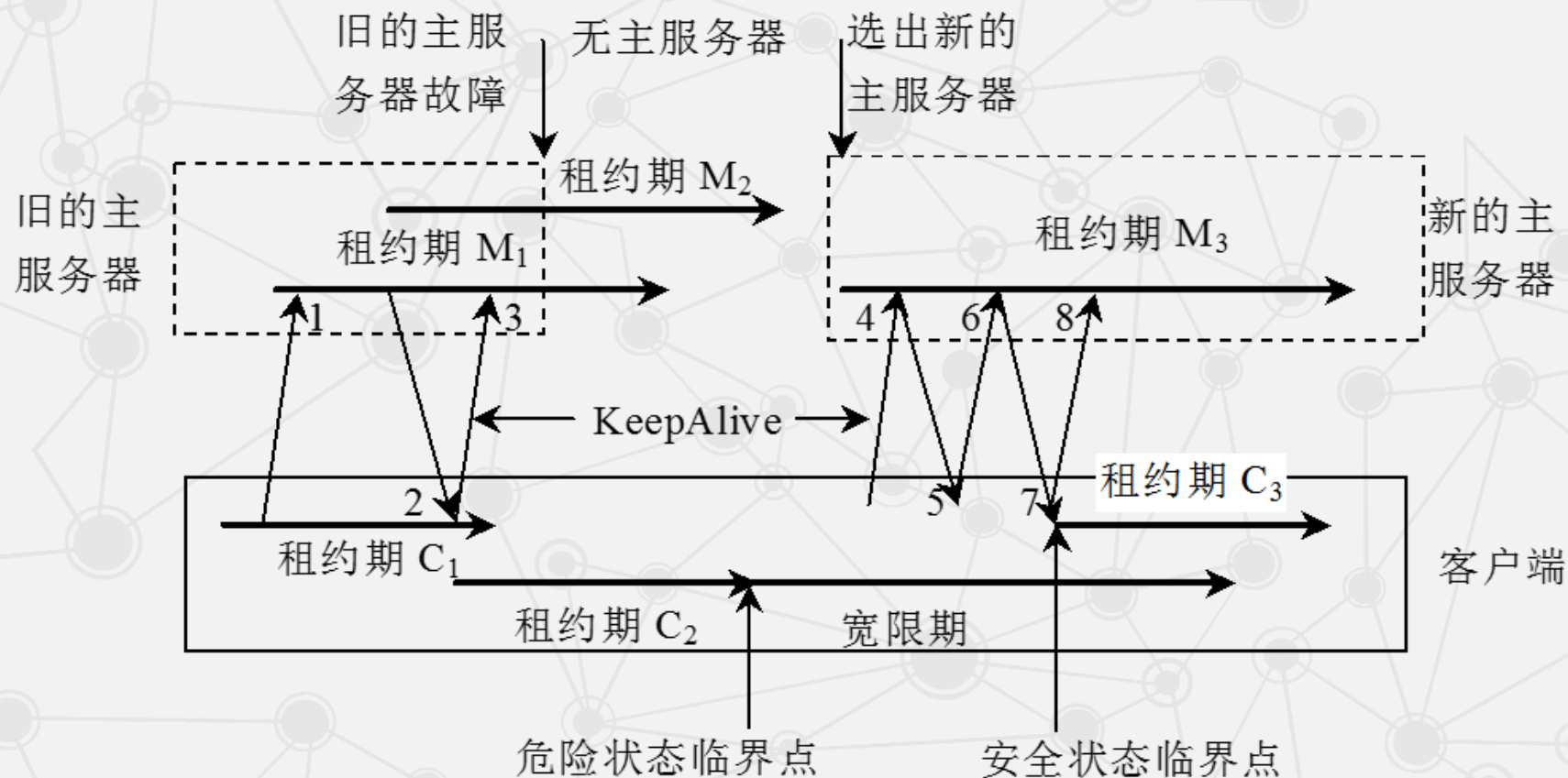


4. 5. 被选出的新主服务器接到第一个KeepAlive请求时，会先**拒绝**，因为这个请求的纪元号错误。

2.3 分布式锁服务Chubby

1. 客户端租约过期

7. 新的主服务器接受这个请求，并立刻做出回应。

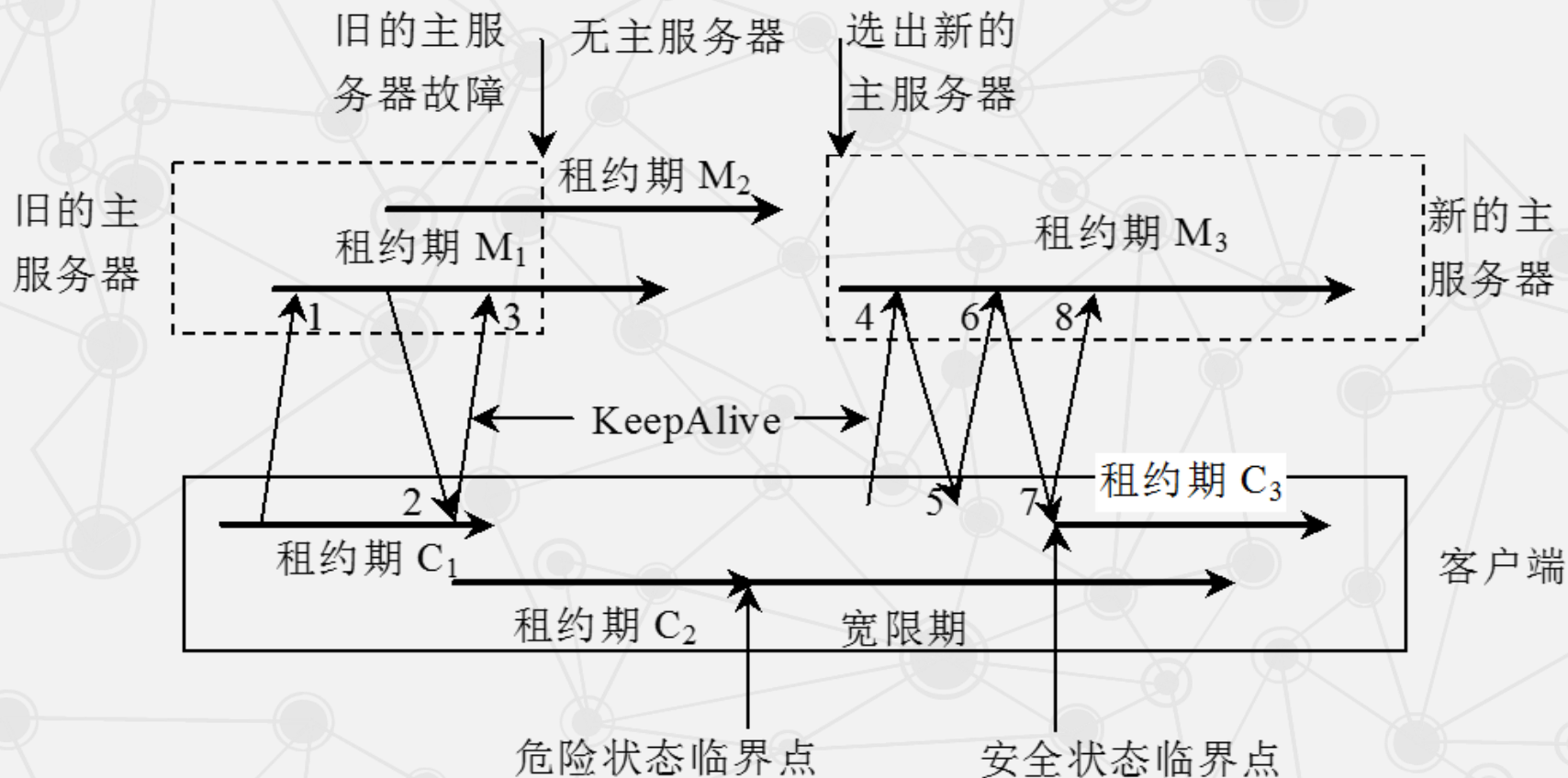


6. 客户端在主服务器拒绝之后，会使用新的纪元号来发送KeepAlive请求。

2.3 分布式锁服务Chubby

● 1. 客户端租约过期

如果在宽限期末接到主服务器的相关回应，客户端终止当前会话。



如果客户端接收到这个回应的时间仍处于宽限期内，系统会恢复到**安全状态**，租约期更新为 C_3 。

2.3 分布式锁服务Chubby

● 2. 主服务器出错

旧服务器出现故障后，系统会很快地选举出新的主服务器，新选举的主服务器在完全运行前需要经历以下9个步骤：

1. 产生一个新的纪元号，以便今后客户端通信时使用，这能保证当前的主服务器不必处理针对旧的主服务器的请求。
2. 只处理与主服务器位置相关的信息，不处理会话相关的信息。
3. 构建处理会话和锁所需的内部数据结构。
4. 允许客户端发送KeepAlive请求，不处理其他会话相关的信息。
5. 向每个会话发送一个故障事件，促使所有的客户端清空缓存。
6. 等待直到所有的会话都收到故障事件或会话终止。
7. 开始允许执行所有的操作。
8. 如果客户端使用了旧的句柄，则需要为其重新构建新的句柄。
9. 一定时间段后（1分钟），删除没有被打开过的临时文件夹。

使用宽限期的好处：如果在宽限期内完成，则用户不会感觉到故障的发生，即新旧服务器的替换对用户来说是透明的，用户仅仅感觉到一个延迟。

2.3 分布式锁服务Chubby

在系统实现时，Chubby还使用了一致性客户端缓存（Consistent Client-Side Caching）技术，其目的是减少通信压力，降低通信频率。

3. 客户端收到这个标志后，会返回一个**确认**（Acknowledge）



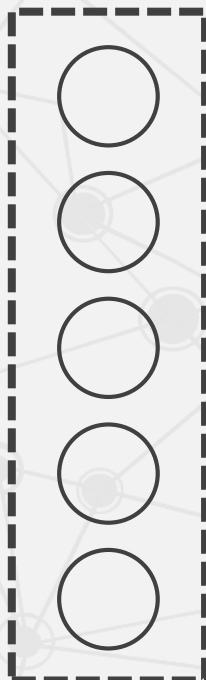
...



客户端进程

远程过程调用

Chubby单元的
五个服务器



主服务器

1. 当某个文件数据或者元数据需要修改时，主服务器首先将这个修改**阻塞**；

2. 通过查询主服务器自身维护的一个缓存表，向对修改的数据进行了缓存的所有客户端发送一个**无效标志**（Invalidation）；

4. 主服务器在收到所有的确认后，才解除阻塞，并完成修改。

这个过程的执行效率非常高，仅仅需要发送一次无效标志即可，因为对于没有返回确认的节点，主服务器直接认为其是未缓存的。

在客户端保存一个和单元上数据一致的本地缓存，需要时，客户可以直接从缓存中取出数据，而不用再和主服务器通信。

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

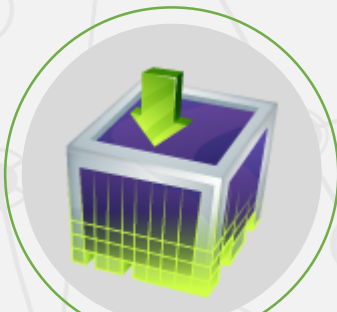
2.3.4 Chubby文件系统

2.3.5 通信协议

▶ 2.3.6 正确性与性能

2.3 分布式锁服务Chubby

- 正确性与性能



一致性



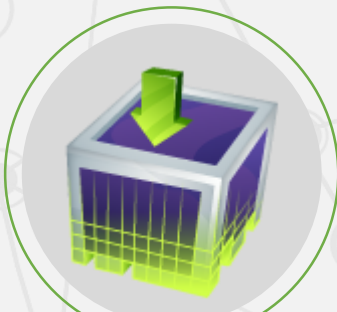
安全性



性能优化

2.3 分布式锁服务Chubby

● 正确性与性能



一致性

每个Chubby单元是由五个副本组成的，这五个副本中需要选举产生一个**主服务器**，这种选举本质上就是一个一致性问题



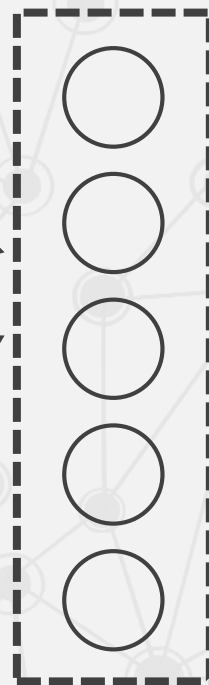
...



客户端进程

远程过程调用

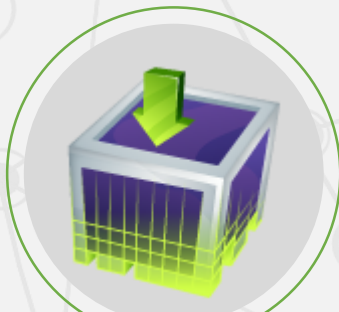
Chubby单元的
五个服务器



主服务器

2.3 分布式锁服务Chubby

● 正确性与性能



一致性

每个Chubby单元是由五个副本组成的，这五个副本中需要选举产生一个**主服务器**，这种选举本质上就是一个一致性问题

在实际的执行过程中，Chubby使用Paxos算法来解决主服务器选举和读写操作时的一致性问题。

读操作很简单，客户只需从主服务器上读取所需的数据。



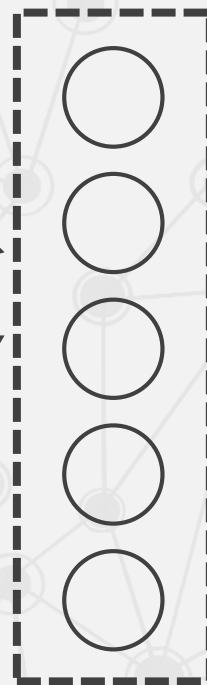
...



客户端进程

远程过程调用

Chubby单元的
五个服务器



主服务器

而写操作就会涉及数据一致性问题，系统就会启用Paxos算法。

2.3 分布式锁服务Chubby

● 正确性与性能

系统中有三种ACL名，分别**写**ACL名（Write ACL Name）、**读**ACL名（Read ACL Name）和**变更**ACL名（Change ACL Name）。



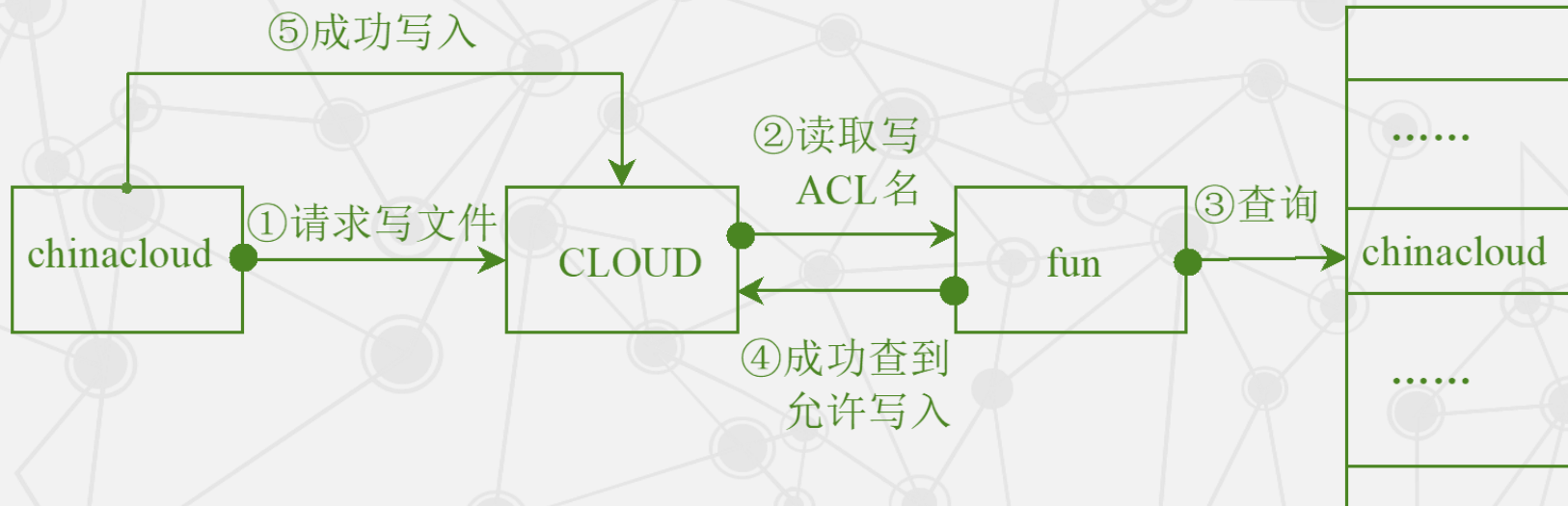
安全性

采用的是ACL形式的安全保障措施。只要不被覆盖，子节点都是直接继承父节点的ACL名。

ACL同样被保存在文件中，是节点**元数据**的一部分。用户在进行相关操作时，首先需要通过ACL来获取相应的授权。

2.3 分布式锁服务Chubby

• Chubby 的 ACL 机制



1. 用户chinacloud提出向文件CLOUD中写入内容的请求。
2. CLOUD首先读取自身的写ACL名fun
3. 接着在fun中查到了chinacloud这一行记录
4. 于是，返回信息，允许chinacloud对文件进行写操作
5. 此时， chinacloud才被允许向CLOUD写入内容。其他的操作和写操作类似。

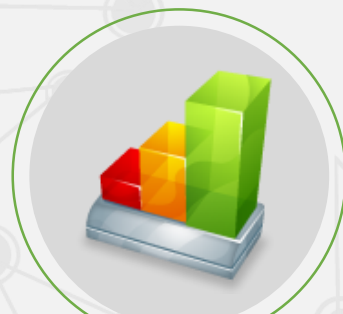
2.3 分布式锁服务Chubby

● 正确性与性能

除此之外，还考虑了使用**代理**（Proxy）和**分区**（Partition）技术。

- **代理**可以**减少**主服务器处理**KeepAlive**以及**读**请求带来的服务器负载，但是**并不能减少写操作**带来的通信量。
Google自己的数据统计表明，在所有请求中，写请求仅占极少一部分，几乎可以忽略不计。
- 使用**分区**技术可以将一个单元的命名空间（Name Space）划成N份。除了少量的跨分区通信外，大部分的分区都可以独自处理服务请求。通过分区，可以**减少**各个分区上的**读写**通信量，但**不能减少KeepAlive**请求的通信量。

因此，将代理和分区结合使用，可以明显提高系统同时处理的服务请求量。



性能优化

为了满足系统的高可扩展性，Chubby采用的措施：**提高主服务器默认的租约期**、使用协议转换服务将Chubby协议转换成较简单的协议、客户端一致性缓存等。